Programa:	Web Boomerang 3.0
Protección:	Visual Protect + Trial 30 días
Objetivos:	Desempakar y evitar que expire
URL descarga:	http://www.filestream.com
Dificultad:	Muy fácil
Herramientas:	OllyDbg v1.10 + ImpRec 1.6 + PeiD 0.94
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 09-11-05

Hola a todos los CracksLatinos ;-)

Hoy atacaremos una protección un tanto desfasada y muy fácil de superar, el nivel exigible para desempakar un Visual Protect es mínimo, su protección se basa en una librería vp.dll donde hace todas las comprobaciones del Time-Trial y demás ya que sólo nos deja 30 días, no tiene stolen bytes y redirige la iat de una manera un tanto pobre diría yo, aún así me he lanzado a escribir el tute, quien sabe quizá haya alguien que esté intentando desempalar un .exe con esta protección y le vaya de perlas ;-DD

Dicho esto vamos al grano, ejecutamos PEiD y miramos a ver que nos dice del ejecutable:

Visual Protect -> Visage

Pues eso Visual Protect, no nos dice de qué versión se trata, RDG packer ni si quiera reconoce el packer que es, algo raro tratándose de una protección que tiene un tiempecito ya, esto nos da igual por ahora sigamos, abrimos un Olly como hacemos siempre y cargamos el ejecutable wbb.exe pues trae otro llamado wbmerang.exe que no nos interesa por el momento, eso sí está igualmente empakado con Visual Protect, quizá al final del tute lo desempakemos para practicar algo más, jeje..

Veamos el Entry Point del packer:

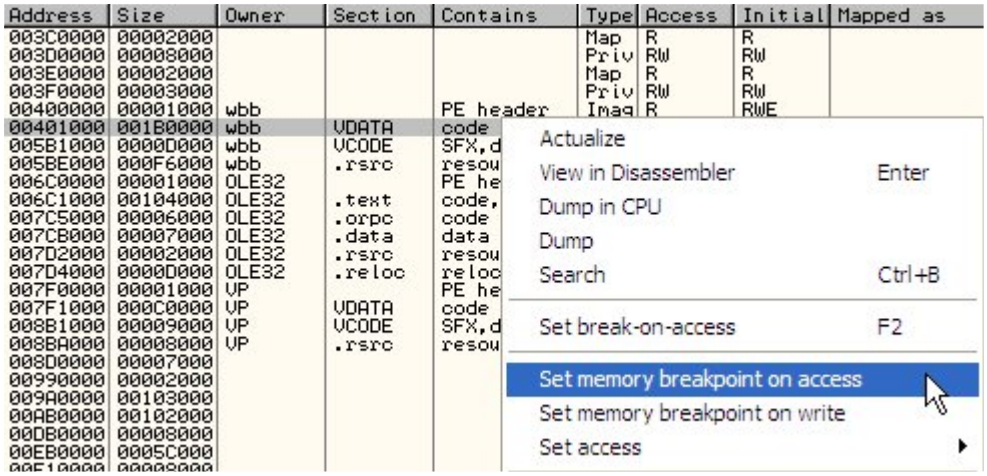
```
005B3F90   55              PUSH EBP
005B3F91   8BEC            MOV EBP,ESP
005B3F93   51              PUSH ECX
005B3F94   53              PUSH EBX
005B3F95   56              PUSH ESI
005B3F96   57              PUSH EDI
005B3F97   C705 B0AE5B00   MOV DWORD PTR DS:[5BAEB0],0
005B3FA1   68 48905B00     PUSH wbb.005B9048                        ASCII "kernel32.dll"
005B3FA6   FF15 00705B00   CALL DWORD PTR DS:[<&KERNEL32.LoadLibraryA>]  kernel32.LoadLibraryA
005B3FAC   A3 0CAF5B00     MOV DWORD PTR DS:[5BAF0C],EAX
005B3FB1   68 58905B00     PUSH wbb.005B9058                        ASCII "GetModuleHandleA"
005B3FB6   A1 0CAF5B00     MOV EAX,DWORD PTR DS:[5BAF0C]
005B3FBB   50              PUSH EAX
005B3FBC   FF15 04705B00   CALL DWORD PTR DS:[<&KERNEL32.GetProcAddress>]  kernel32.GetProcAddress
005B3FC2   A3 90AE5B00     MOV DWORD PTR DS:[5BAE90],EAX
005B3FC7   6A 00           PUSH 0
005B3FC9   FF15 90AE5B00   CALL DWORD PTR DS:[5BAE90]
005B3FCF   A3 ECAE5B00     MOV DWORD PTR DS:[5BAEEC],EAX
005B3FD4   8B0D ECAE5B00   MOV ECX,DWORD PTR DS:[5BAEEC]
005B3FDA   51              PUSH ECX
005B3FDB   E8 C0050000     CALL wbb.005B45A0
005B3FE0   83C4 04         ADD ESP,4
005B3FE3   8945 FC         MOV DWORD PTR SS:[EBP-4],EAX
005B3FE6   837D FC 00      CMP DWORD PTR SS:[EBP-4],0
005B3FEA   74 03           JE SHORT wbb.005B3FEF
005B3FEC   FF65 FC         JMP DWORD PTR SS:[EBP-4]
005B3FEF   5F              POP EDI
005B3FF0   5E              POP ESI
005B3FF1   5B              POP EBX
005B3FF2   8BE5            MOV ESP,EBP
005B3FF4   5D              POP EBP
005B3FF5   C3              RETN
```

Estamos en el inicio de código, el programa comenzará a ejecutarse a partir de aquí, o sea que el tema de la descompresión debe de hacerse en alguno de los CALl que hay antes del RET, en concreto creo que lo hace en :5B3FD8 por lo que he podido ver, lo primero que haremos como siempre es averiguar el OEP del programa, o sea el punto de entrada real, el que tenía antes de empaketarlo con Visual Protect, para descubrirlo existen varios métodos aunque uno de los más fiables es el de Narvaja, que consiste en ir superando las excepciones que se vayan sucediendo y anotar la última, en este caso es:

```
007F478F   3A0A            CMP CL,BYTE PTR DS:[EDX]
007F4791   74 17           JE SHORT VP.007F47AA
007F4793   3A4A 01         CMP CL,BYTE PTR DS:[EDX+1]
007F4796   74 11           JE SHORT VP.007F47A9
007F4798   3A4A 02         CMP CL,BYTE PTR DS:[EDX+2]
007F479B   74 0B           JE SHORT VP.007F47A8
007F479D   3A4A 03         CMP CL,BYTE PTR DS:[EDX+3]
```

después de esta ya se ejcuta el programa, repetimos el proceso pero antes de superar esta excepción, he de decir que entre tanta excepción que hay unas cuantas nos aparece la nag del programa:

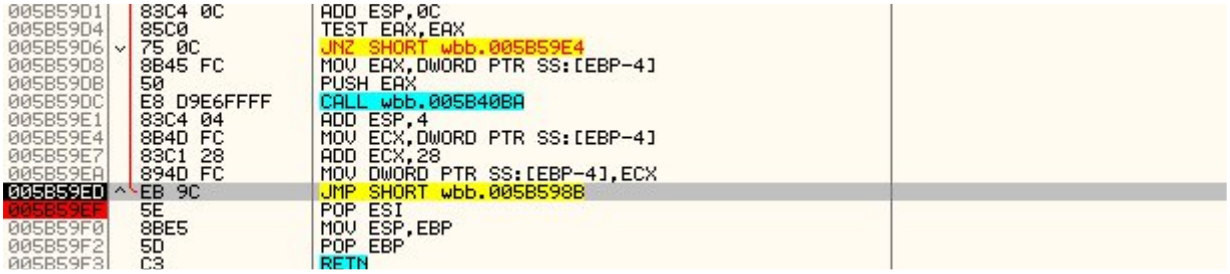
elegimos "Try" y continúan las excepciones, llegamos a la última que es la que hemos visto antes y ponemos el BPM en la primera sección:



Esto está más que explicado pero como hemos visto últimamente hay gente que empieza ahora y no lo sabe, por esto no está de más explicarlo de nuevo, puesto el BPM ahora sí superamos la excepción con Shift+F9 y para donde no nos interesa:

Sabemos que no es la zona porque paró por escritura como vemos:

Allí nos lo está diciendo Olly que paró por escritura y no por ejecución, traceamos un poco y vemos que estamos en un bucle, salimos aquí:



Ponemos el BP donde la imagen :5B59EF y quitamos el BPM para que no moleste, pulsamos F9 y para en el BP que quitamos de inmediato, volvemos a poner el BPM y vuelve a parar por escritura, vamos deshaciendo los bucles que nos salgan hasta que pare en el OEP:

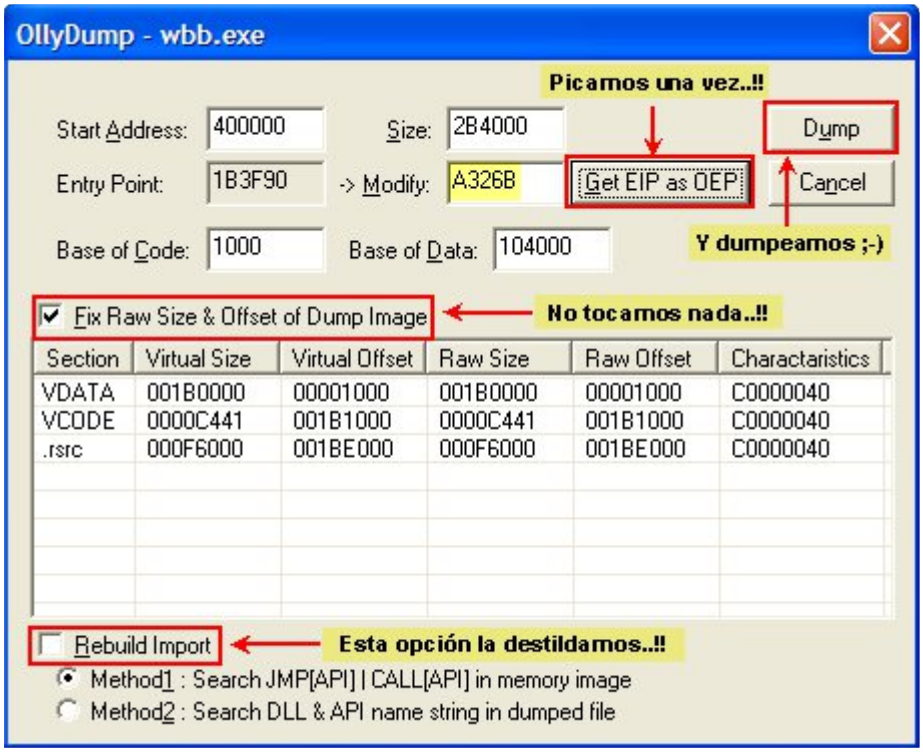
```
00895FC7   B8 08000000   MOV EAX,8
00895FCC   E8 AF2FF6FF   CALL VP.007F8F80
00895FD1   8B8D 58FAFFFF MOV ECX,DWORD PTR SS:[EBP-5A8]
00895FD7   8D85 5CFAFFFF LEA EAX,DWORD PTR SS:[EBP-5A4]
00895FDD   BA 946B8900   MOV EDX,VP.00896B94                ASCII "Finalizing 0x"
00895FE2   E8 B5E8F5FF   CALL VP.007F489C
00895FE7   8B85 5CFAFFFF MOV EAX,DWORD PTR SS:[EBP-5A4]
00895FED   E8 46BCFFFF   CALL VP.00891C38
00895FF2 - FF65 FC       JMP DWORD PTR SS:[EBP-4]           wbb.004A326B
00895FF5   6A 00         PUSH 0
00895FF7   E8 C00CF6FF   CALL VP.007F6CBC
00895FFC   E9 C6040000   JMP VP.008964C7
00896001   A1 70BE8900   MOV EAX,DWORD PTR DS:[89BE70]
```

En realidad ese es el salto que descubrimos si traceamos la zona en lugar de utilizar el método de las excepciones comentado, igual da el resultado es este:

```
004A326B   55            PUSH EBP          <-- el OEP..!!
004A326C   8BEC          MOV EBP,ESP
004A326E   6A FF         PUSH -1
004A3270   68 08FC5200   PUSH wbb.0052FC08
004A3275   68 78164A00   PUSH wbb.004A1678
004A327A   64:A1 00000000 MOV EAX,DWORD PTR FS:[0]
004A3280   50            PUSH EAX
004A3281   64:8925 000000( MOV DWORD PTR FS:[0],ESP
004A3288   83EC 58       SUB ESP,58
004A328B   53            PUSH EBX
004A328C   56            PUSH ESI
004A328D   57            PUSH EDI
004A328E   8965 E8       MOV DWORD PTR SS:[EBP-18],ESP
004A3291   FF15 60445000 CALL DWORD PTR DS:[504460]  <-- Aqui problemas..!!
004A3297   33D2          XOR EDX,EDX
```

A veces te mete en una excepción no continuable ¿? Pero al final ha tenido recompensa, llegamos al OEP del programa, este es el mejor punto para dumpearlo ;-)

Venga vamos…



Dumpeamos con estas opciones, el plugin Ollydump va muy bien, lo guardamos como tute.exe para variar, jeje.. y ahora a por la IAT…

Lo primero es seguirle la pista a un puntero inválido, por ejemplo este:

```
004A3291    FF15 60445000    CALL DWORD PTR DS:[504460]
```

Que es el primero que nos encontramos bajando desde el OEP, nos situamos sobre él y pulsamos con el botón derecho, se desplega un menú flotante donde escogemos la opción "Follow in dump->Memory address" para aterrirzar en plena IAT, a partir de aquí buscamos el inicio y el final de la misma subiendo y bajando con los cursores:

```
Address   Hex dump                                          ASCII
00503FE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00503FF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00504000  D8 17 DA 77 BB 28 DA 77 68 6A DB 77 F4 6C DB 77  ï‡rw╗(rwhj■w¶l■w
00504010  9A 22 DA 77 27 67 DB 77 63 51 DA 77 B1 63 DB 77  ü"rw'g■wcQrw▒c■w
00504020  D4 65 DB 77 D9 23 DA 77 5B 66 DB 77 D6 27 DA 77  ée■w┘#rw[f■wí'rw
00504030  E2 68 DB 77 69 6D DC 77 8E 5B DC 77 4D 22 DB 77  ôh■wim▄wÄ[▄wM"■w
```

comienza en :504000 como vemos en la imagen anterior, hagamos lo mismo para el final:

```
Address   Hex dump                                          ASCII
005049C0  F4 83 6D 00 C3 EA 6E 00 60 85 6D 00 AF 9F 70 00  ¶âm.├ön.`àm.»ƒp.
005049D0  19 86 6D 00 41 07 6D 00 F8 BA 72 00 A1 D0 71 00  ↓åm.A•m.°╢r.íðq.
005049E0  44 30 73 00 CF 94 6D 00 C3 F3 6F 00 61 EE 6E 00  D0s.Ïöm.├óo.aîn.
005049F0  6F A9 71 00 F3 A4 71 00 AA 90 71 00 A9 48 6D 00  o©q.¾ñq.¬Éq.®Hm.
00504A00  5A A2 70 00 53 07 6D 00 9F E1 6E 00 00 00 00 00  Zóp.S•m.ƒßn.....
00504A10  F3 F0 CC 74 00 00 00 00 00 00 00 00 00 00 00 00  ¾-╠t............
00504A20  A8 89 52 00 28 4A 50 00 00 00 00 00 00 00 00 00  ¿ëR.(JP.........
00504A30  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

termina en :504A10 pero al ImpRec le daremos como final :504A14 donde empieza a haber ceros (zona sombreada en verde), anotamos ambos valores y sacamos el largo fácilmente:

504A14 - 504000 = A14

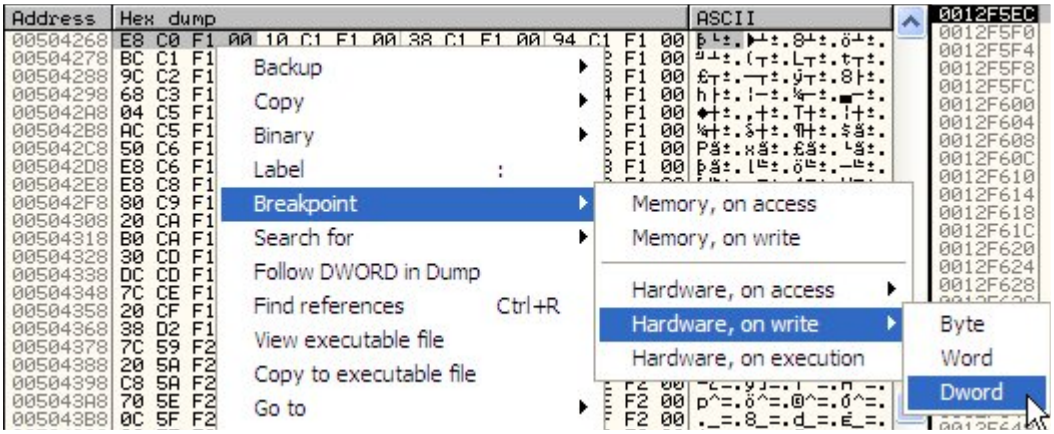
Una vez tenemos la IAT bien definida vamos a tratar de encontrar el salto mágico que la mayoría de packers tienen, lo necesitamos para evitar que nos redirija parte de la iat:

```
Address   Hex dump                                          ASCII
005041A0  00 42 C4 77 9D 50 C4 77 10 85 C4 77 2B A0 C5 77  .B─w¥P─w►à─w+á┼w
005041B0  A7 51 C4 77 76 80 C4 77 33 83 C4 77 BF 21 C4 77  §Q─wvÇ─w3â─w┐!─w
005041C0  90 84 C4 77 D4 84 C4 77 A8 62 C4 77 2F 52 C4 77  Éä─wÉä─w¿b─w/R─w
005041D0  B3 D3 C5 77 25 13 C7 77 92 CE C4 77 55 AD C4 77  │É┼w%‼╟w╬─wU¡─w
005041E0  E9 35 C5 77 78 23 C4 77 C7 5A C4 77 EE 63 C4 77  ú5┼wx#─w╟Z─w╦c─w
005041F0  D1 36 C4 77 6E 3C C4 77 0E 6C C4 77 5B 58 C4 77  Ð6─wn<─w♫l─w[X─w
00504200  9B 40 C4 77 71 4B C4 77 3D 51 C4 77 9D 88 C4 77  ¢@─wqK─w=Q─w¥ê─w
00504210  DC 7E C4 77 64 3B C5 77 07 36 C4 77 E0 20 C4 77  ▄~─wd;┼w•6─wó ─w
00504220  1A 1E C4 77 46 D8 C4 77 06 8A C4 77 F4 69 C7 77  →▲─wFï─w♠è─w¶i╟w
00504230  0B ED C6 77 AE 5A C4 77 6A 3D C4 77 67 47 C4 77  ♂ÿ╞w«Z─wj=─wgG─w
00504240  7C 21 C4 77 22 4F C4 77 FE 93 C4 77 8E 21 C4 77  |!─w"O─w■ô─wÄ!─w
00504250  9F 86 C4 77 98 1B C4 77 4D 83 C4 77 5F 3A C4 77  ƒå─wÿ←─wMâ─w_:─w
00504260  6C 5B C4 77 00 00 00 00 E8 C0 F1 00 10 C1 F1 00  l[─w....è└±.►┴±.
00504270  38 C1 F1 00 94 C1 F1 00 BC C1 F1 00 28 C2 F1 00  8┴±.ö┴±.╝┴±.(┬±.
00504280  4C C2 F1 00 74 C2 F1 00 9C C2 F1 00 C4 C2 F1 00  L┬±.t┬±.£┬±.─┬±.
00504290  EC C2 F1 00 38 C3 F1 00 68 C3 F1 00 7C C4 F1 00  ╦┬±.8├±.h├±.│─±.
005042A0  AC C4 F1 00 DC C4 F1 00 04 C5 F1 00 2C C5 F1 00  ¼─±.▄─±.♦┼±.,┼±.
005042B0  54 C5 F1 00 7C C5 F1 00 AC C5 F1 00 D0 C5 F1 00  T┼±.│┼±.¼┼±.Ð┼±.
005042C0  F4 C5 F1 00 24 C6 F1 00 50 C6 F1 00 78 C6 F1 00  ¶┼±.$ã±.Pã±.xã±.
```

los punteros sombreados apuntan todos a zonas del tipo :F1Cxxx que nuestro dumpeado no tendrá claro, esto es porque el packer ha creado una sección en tiempo de ejecución donde ha insertado código y ha redireccionado algunos punteros de la iat hacia esta zona, encontrar el salto mágico nos va a ayudar para impedir que lo haga, comenzaremos centrándonos en una entrada MALA como esta:

```
Address   Hex dump                                          ASCII
00504268  E8 C0 F1 00 10 C1 F1 00 38 C1 F1 00 94 C1 F1 00  è└±.►┴±.8┴±.ö┴±.
00504278  BC C1 F1 00 28 C2 F1 00 4C C2 F1 00 74 C2 F1 00  ╝┴±.(┬±.L┬±.t┬±.
00504288  9C C2 F1 00 C4 C2 F1 00 EC C2 F1 00 38 C3 F1 00  £┬±.─┬±.╦┬±.8├±.
00504298  68 C3 F1 00 7C C4 F1 00 AC C4 F1 00 DC C4 F1 00  h├±.│─±.¼─±.▄─±.
```

Como digo nos centramos en esta entrada resaltada [504268]==F1C0E8, le ponemos un "BPHardware en Memory Write" de esta manera:



Y reiniciamos Olly con Ctrl+F2, tildamos todas las excepciones en "Options->Debugging options" para que Olly no pare en ninguna, pues sólo queremos parar cuando escriba en la entrada de la IAT en busca del lugar donde guarda los bytes finales E8C0F100, una vez encontrado el punto será cuestion de traceo en busca del mágico como hacemos

siempre, una vez reiniciado Olly y estando en el EntryPoint pulsamos F9, como tenemos el HBP puesto no hay problema, sale la nag pulsamos Try y para aquí:

```
005B623E   46          INC ESI
005B623F   8807        MOV BYTE PTR DS:[EDI],AL
005B6241   47          INC EDI                          wbb.00504268
005B6242   00D2        ADD DL,DL
005B6244 v 75 05       JNZ SHORT wbb.005B624B
```

Pero no escribe los bytes finales, seguimos con F9 y como va copiando byte a byte para 4 veces, a la quinta para en otra parte:

```
005B62E4   F3:A4       REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
005B62E6   5E          POP ESI
005B62E7 ^ E9 56FFFFFF JMP wbb.005B6242
005B62EC   48          DEC EAX
005B62ED   C1E0 08     SHL EAX,8
005B62F0   8A06        MOV AL,BYTE PTR DS:[ESI]
```

típico REP MOVS que nunca falta, jeje… y a la tercera va la vencida ;-)

```
0088044C   E8 63FDFFFF CALL VP.008801B4
00880451   8BD8        MOV EBX,EAX
00880453   8B07        MOV EAX,DWORD PTR DS:[EDI]
00880455   8918        MOV DWORD PTR DS:[EAX],EBX       ;aqui guarda...
00880457   85DB        TEST EBX,EBX
00880459 v 75 0E       JNZ SHORT VP.00880469
0088045B   8B45 08     MOV EAX,DWORD PTR SS:[EBP+8]
0088045E   8B40 E4     MOV EAX,DWORD PTR DS:[EAX-1C]
```

Se detiene justo donde nos interesa, EAX tiene la posición de la IAT donde se escribirá y EBX los bytes finales que allí quedan y que antes vimos en el OEP, veámoslo mejor:

```
Registers (FPU)
EAX 00504268 wbb.00504268
ECX 77E40000 kernel32.77E40000
EDX 0012F5EC
EBX 00F144BC
ESP 0012F5A0
EBP 0012F5AC
ESI 0012F5D8
EDI 0012F5D4 ASCII "hBP"
```

Además ECX almacena la imagebase del Kernel así que imagino está tratando punteros de esta DLL, a partir de aquí el trabajo es sencillo, cuestión de tracear la rutina en busca del salto que evite que se redireccionen las entradas, de momento ponemos un HBP en la dirección donde guarda:

```
00880455    8918            MOV DWORD PTR DS:[EAX],EBX
```

Este nos ayudará si tenemos que reiniciar o algo así, además dejamos uno siempre en el OEP por el mismo motivo pues no está de más, el final del bucle donde trata la iat es:

```
00880461   83C0 02     ADD EAX,2
00880464   E8 EBFDFFFF CALL VP.00880254
00880469   8306 04     ADD DWORD PTR DS:[ESI],4
0088046C   8307 04     ADD DWORD PTR DS:[EDI],4
0088046F   8B06        MOV EAX,DWORD PTR DS:[ESI]
00880471   8B18        MOV EBX,DWORD PTR DS:[EAX]
00880473   85DB        TEST EBX,EBX
00880475 ^ 0F85 61FFFFFF JNZ VP.008803DC
0088047B   5F          POP EDI
0088047C   5E          POP ESI
0088047D   5B          POP EBX
0088047E   5D          POP EBP
0088047F   C3          RETN
```

terminará cuando EBX==0 y este toma el valor de [EAX] que lógicamente va cambiando, si seguimos el salto (JNZ 8803DC) vamos a lo que es el inicio del mismo:

```
008803DC   8BC3        MOV EAX,EBX
008803DE   E8 51FEFFFF CALL VP.00880234
008803E3   84C0        TEST AL,AL
008803E5 v 74 3C       JE SHORT VP.00880423
008803E7   8B06        MOV EAX,DWORD PTR DS:[ESI]
008803E9   8B00        MOV EAX,DWORD PTR DS:[EAX]
```

Y ya tenemos bien delimitada esta zona, lo siguiente lógicamente es encontrar el mágico, la manera que yo tengo de hacerlo e imagino que la de muchos es buscar el call GetProcAddress, lo llamo así porque en realidad pone la API en EAX que es lo que hace GetProcAddress, una vez localizado este CALL sólo me debo fijar si lo cambia por un valor MALO antes de llegar a la zona donde guarda que recordemos que era esta:

```
00880455    8918            MOV DWORD PTR DS:[EAX],EBX
```

Aquí EBX tiene el valor que irá en la IAT y EAX tiene la dirección donde irá, o sea que representa una posición dentro de la IAT, en concreto [EAX] que es el puntero ok? Sólo

debemos localizar el call GetProcAddress y un HBP en la instrucción donde guarda los valores y estudiar el comportamiento cuando guarda una buena de cuando la guarda mala, si traceamos un poquitín enseguida descubrimos el citado CALL, veamos una panorámica:

```
008803DC  8BC3          MOV EAX,EBX                          ;inicio_loop..
008803DE  E8 51FEFFFF   CALL VP.00880234
008803E3  84C0          TEST AL,AL
008803E5  74 3C         JE SHORT VP.00880423
008803E7  8B06          MOV EAX,DWORD PTR DS:[ESI]
008803E9  8B00          MOV EAX,DWORD PTR DS:[EAX]
008803EB  83E8 0A       SUB EAX,0A
008803EE  E8 4DFEFFFF   CALL VP.00880240
008803F3  8B55 08       MOV EDX,DWORD PTR SS:[EBP+8]
008803F6  8942 EC       MOV DWORD PTR DS:[EDX-14],EAX
008803F9  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
008803FC  8B40 EC       MOV EAX,DWORD PTR DS:[EAX-14]
008803FF  50            PUSH EAX
00880400  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00880403  8B40 E8       MOV EAX,DWORD PTR DS:[EAX-18]
00880406  50            PUSH EAX
00880407  E8 D4FCFFFF   CALL VP.008800E0
0088040C  8BD8          MOV EBX,EAX
0088040E  8B07          MOV EAX,DWORD PTR DS:[EDI]
00880410  8918          MOV DWORD PTR DS:[EAX],EBX
00880412  85DB          TEST EBX,EBX
00880414  75 53         JNZ SHORT VP.00880469
00880416  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00880419  8B40 EC       MOV EAX,DWORD PTR DS:[EAX-14]
0088041C  E8 33FEFFFF   CALL VP.00880254
00880421  EB 46         JMP SHORT VP.00880469
00880423  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00880426  50            PUSH EAX
00880427  8B06          MOV EAX,DWORD PTR DS:[ESI]
00880429  8B00          MOV EAX,DWORD PTR DS:[EAX]
0088042B  E8 18FEFFFF   CALL VP.00880248
00880430  59            POP ECX
00880431  8B55 08       MOV EDX,DWORD PTR SS:[EBP+8]
00880434  8942 E4       MOV DWORD PTR DS:[EDX-1C],EAX
00880437  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
0088043A  8B48 E0       MOV ECX,DWORD PTR DS:[EAX-20]        ;ECX apunta a la DLL...
0088043D  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00880440  8B40 E4       MOV EAX,DWORD PTR DS:[EAX-1C]
00880443  8D50 02       LEA EDX,DWORD PTR DS:[EAX+2]
00880446  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
00880449  8B40 E8       MOV EAX,DWORD PTR DS:[EAX-18]        ;en EAX la imagebase de la DLL...
0088044C  E8 63FDFFFF   CALL VP.008801B4                     ;call GetProcAddress, jeje...
00880451  8BD8          MOV EBX,EAX
00880453  8B07          MOV EAX,DWORD PTR DS:[EDI]           ;en EAX la posicion de la IAT...
00880455  8918          MOV DWORD PTR DS:[EAX],EBX           ;y aqui guarda la API...
00880457  85DB          TEST EBX,EBX
00880459  75 0E         JNZ SHORT VP.00880469
0088045B  8B45 08       MOV EAX,DWORD PTR SS:[EBP+8]
0088045E  8B40 E4       MOV EAX,DWORD PTR DS:[EAX-1C]
00880461  83C0 02       ADD EAX,2
00880464  E8 EBFDFFFF   CALL VP.00880254
00880469  8306 04       ADD DWORD PTR DS:[ESI],4
0088046C  8307 04       ADD DWORD PTR DS:[EDI],4             ;apunta al sgute DWORD de la IAT...
0088046F  8B06          MOV EAX,DWORD PTR DS:[ESI]
00880471  8B18          MOV EBX,DWORD PTR DS:[EAX]
00880473  85DB          TEST EBX,EBX
00880475  0F85 61FFFFFF JNZ VP.008803DC                      ;fin_loop..
0088047B  5F            POP EDI
0088047C  5E            POP ESI
0088047D  5B            POP EBX
0088047E  5D            POP EBP
0088047F  C3            RETN
```

Este es el bucle donde va tratando todas las entradas de la IAT, sabemos que algunas las redirige y otras no, necesitamos encontrar el salto que decide si redirigirla o no (salto mágico) he puesto algunos comentarios, lo más interesante por ahora está en la dirección :88044C que es donde pone la API en EAX ;-)

Lo que haremos será poner un HBP en dicha dirección (no pondremos BP's de momento por si acaso) y entrar en el CALL cuando lleguemos a él, vemos que no es muy grande, apenas 6 CALL's y algunas instrucciones, lo que haremos será tracear el código en busca de algo, como no se ve ningún salto suponemos que estará dentro de un subcall, sólo tiene 6 así que la cosa está fácil, veamos la zona:

```
008801F3  E8 2082FDFF   CALL VP.00858418
008801F8  8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
008801FB  E8 4848F7FF   CALL VP.007F4A48                     ;en EAX el nombre de la API...
00880200  50            PUSH EAX
00880201  8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]         ;en EAX la base de la DLL...
00880204  50            PUSH EAX
00880205  E8 D6FEFFFF   CALL VP.008800E0                     ;call GetProcAddress...
0088020A  8BD8          MOV EBX,EAX
0088020C  33C0          XOR EAX,EAX
0088020E  5A            POP EDX
```

donde está puesto el BP llama a GetProcAddress, como hemos dicho antes no vemos ningún salto así que estará dentro de este CALL, la próxima vez que lleguemos a él entramos, recordad que estamos en un bucle así que pasaremos más veces, una vez dentro vemos una zona llena de saltos y comprobaciones, un poco más abajo la famosa llamada a GetProcAddress, jejeje… nos vamos acercando amigos, de momento traceamos línea a línea con F8 para ver cómo se comporta y por donde tira, después nos fijarmemos si guarda una entrada BUENA o una MALA y así sabremos por donde atacar ;-)

Veamos una imagen también de este subcall que me parece interesante:

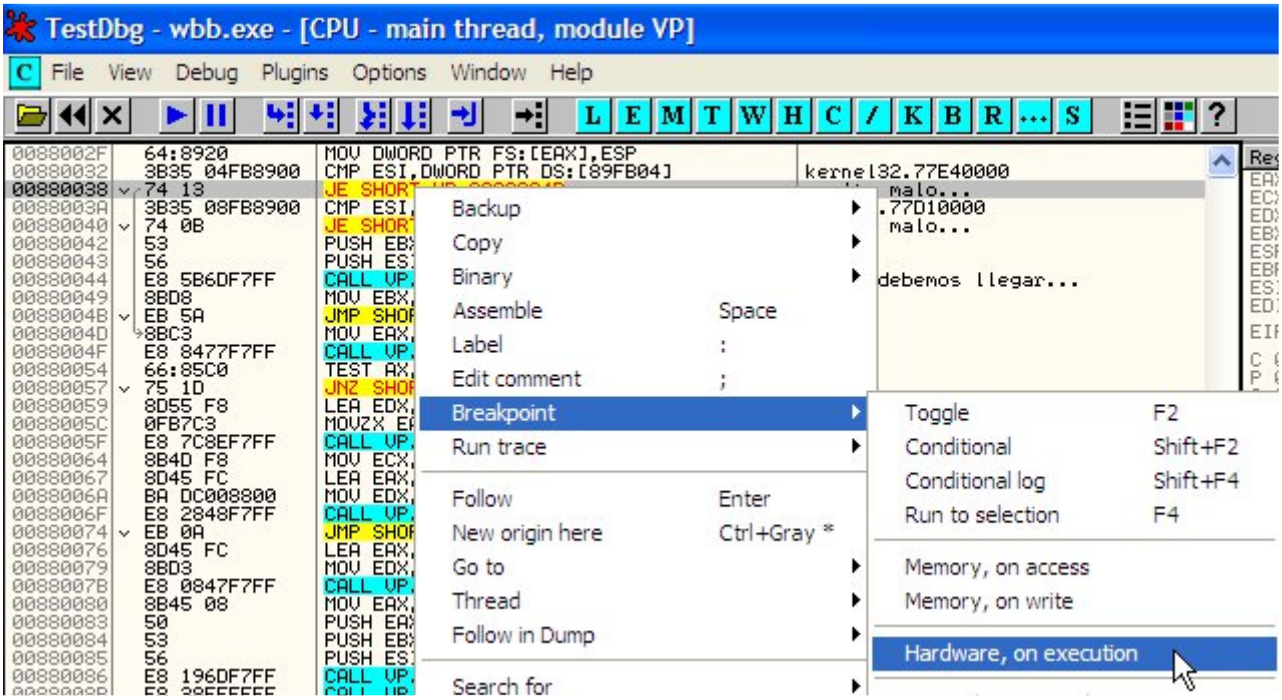
```
00880149   E8 4289F7FF   CALL VP.007F8A90
0088014E   84C0          TEST AL,AL
00880150 v 74 0F         JE SHORT VP.00880161
00880152   3B35 04FB8900 CMP ESI,DWORD PTR DS:[89FB04]      kernel32.77E40000
00880158 v 75 07         JNZ SHORT VP.00880161
0088015A   BB E0008800   MOV EBX,VP.008800E0
0088015F v EB 12         JMP SHORT VP.00880173
00880161   55            PUSH EBP
00880162   8BCB          MOV ECX,EBX
00880164   8BD6          MOV EDX,ESI
00880166   A1 14FB8900   MOV EAX,DWORD PTR DS:[89FB14]
0088016B   E8 A4FEFFFF   CALL VP.00880014                   ;call GetProcAddress...
00880170   59            POP ECX
00880171   8BD8          MOV EBX,EAX
```

Pongo solamente lo más interesante sino no acabamos nunca, si miramos por encima vemos referencias claras a la api GetProcAddress pero todavía no hemos dado con lo que buscamos, ponemos un BP en el CALL como en la imagen y la próxima vez que pare entramos dentro con F7 a ver qué encontramos:

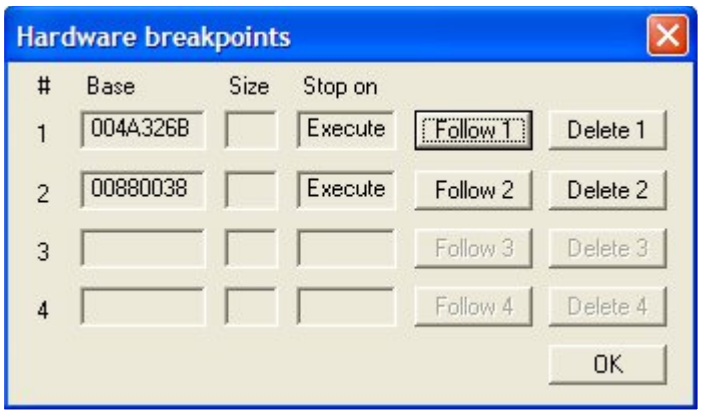
```
00880026   55            PUSH EBP
00880027   68 C2008800   PUSH VP.008800C2
0088002C   64:FF30       PUSH DWORD PTR FS:[EAX]
0088002F   64:8920       MOV DWORD PTR FS:[EAX],ESP
00880032   3B35 04FB8900 CMP ESI,DWORD PTR DS:[89FB04]      kernel32.77E40000
00880038 v 74 13         JE SHORT VP.0088004D               ;salto malo...
0088003A   3B35 08FB8900 CMP ESI,DWORD PTR DS:[89FB08]      USER32.77D10000
00880040 v 74 0B         JE SHORT VP.0088004D               ;salto malo...
00880042   53            PUSH EBX
00880043   56            PUSH ESI
00880044   E8 5B6DF7FF   CALL VP.007F6DA4                   ;Aqui debemos llegar...
00880049   8BD8          MOV EBX,EAX
0088004B v EB 5A         JMP SHORT VP.008800A7
0088004D   8BC3          MOV EAX,EBX
0088004F   E8 8477F7FF   CALL VP.007F77D8
00880054   66:85C0       TEST AX,AX
```

esto sí es lo que buscábamos, dos saltos juntos, hmmmm... esto huele a mágicos ¿tendrá dos quizá? Lo veremos, lo que más me choca es que mira que DLL está tatando y en caso de ser la kernel32 o la user32 se produce el salto a una zona donde nos pone un valor MALO en EAX, en cambio si no salta ninguno de los dos llega al CALL en :880044 donde pone la API correcta ;-)

Chicos tenemos el mágico, jeje.. bueno los mágicos pues son dos en este caso y es una de las pocas veces que lo veo, casi siempre suele haber uno pero dos? jeje.. bueno probaremos a reparar la IAT mediante estos dos saltos, lo primero que se me ocurre es quitar todos los BP's que tengamos y los HE's también excepto el del OEP para que pare allí cuando tengamos la IAT reparada, veremos si funciona:



Una vez que lo hemos puesto nos aseguramos de tener el otro en el OEP como dije antes, si no lo hacemos el programa echará a correr una vez hayamos reparado la IAT y no podremos dumpearlo, para evitar esto sólo debemos poner otro HBP en dicho OEP, veamos en una imagen la ventanita de HBP's para que se vea con claridad como lo debemos tener:



Esos son los únicos dos que debemos tener, ahora si no hemos reiniciado Olly lo hacemos y pulsamos F9 para detenernos en el primero de los saltos mágicos y nopeamos ambos:

```
00880038   90              NOP                                  ;salto malo...
00880039   90              NOP
0088003A   3B35 08FB8900   CMP ESI,DWORD PTR DS:[89FB08]        USER32.77D10000
00880040   90              NOP                                  ;salto malo...
00880041   90              NOP
00880042   53              PUSH EBX
00880043   56              PUSH ESI
00880044   E8 5B6DF7FF     CALL VP.007F6DA4                     ;Aqui debemos llegar...
00880049   8BD8            MOV EBX,EAX
0088004B   EB 5A           JMP SHORT VP.008800A7
```

Y como tenemos un HBP en el OEP damos RUN (F9) para que corra y se detenga en el OEP que estaba recordemos en :4A326B allí le echamos un vistazo a la IAT a ver cómo quedó, pero antes debemos quitar el HBP de la dirección del primer mágico porque sinó se va parando en cada entrada y se hace muy pesado, ahora sí veamos la IAT:

```
Address   Hex dump                                          ASCII
00504460  58 D5 16 00 78 D5 16 00 98 D5 16 00 B8 D5 16 00  X'..x'..ÿ'..@'..
00504470  D8 D5 16 00 F8 D5 16 00 18 D6 16 00 38 D6 16 00  ï'..°'..↑ï..8ï..
00504480  58 D6 16 00 78 D6 16 00 98 D6 16 00 B8 D6 16 00  Xï..xï..ÿï..@ï..
00504490  D8 D6 16 00 F8 D6 16 00 18 D7 16 00 38 D7 16 00  ïï..°ï..↑ï..8ï..
005044A0  58 D7 16 00 78 D7 16 00 98 D7 16 00 B8 D7 16 00  Xï..xï..ÿï..@ï..
005044B0  D8 D7 16 00 F8 D7 16 00 18 D8 16 00 38 D8 16 00  ïï..°ï..↑ï..8ï..
005044C0  58 D8 16 00 78 D8 16 00 98 D8 16 00 B8 D8 16 00  Xï..xï..ÿï..@ï..
005044D0  D8 D8 16 00 F8 D8 16 00 18 D9 16 00 38 D9 16 00  ïï..°ï..↑┘..8┘..
005044E0  58 D9 16 00 78 D9 16 00 98 D9 16 00 B8 D9 16 00  X┘..x┘..ÿ┘..@┘..
005044F0  D8 D9 16 00 F8 D9 16 00 18 DA 16 00 38 DA 16 00  ï┘..°┘..↑┌..8┌..
00504500  58 DA 16 00 78 DA 16 00 98 DA 16 00 B8 DA 16 00  X┌..x┌..ÿ┌..@┌..
00504510  D8 DA 16 00 F8 DA 16 00 18 DB 16 00 38 DB 16 00  ï┌..°┌..↑█..8█..
00504520  58 DB 16 00 78 DB 16 00 98 DB 16 00 B8 DB 16 00  X█..x█..ÿ█..@█..
00504530  D8 DB 16 00 F8 DB 16 00 18 DC 16 00 38 DC 16 00  ï█..°█..↑▄..8▄..
00504540  58 DC 16 00 78 DC 16 00 98 DC 16 00 00 00 00 00  X▄..x▄..ÿ▄.....
00504550  DA 3F B5 73 19 22 B5 73 20 23 B5 73 83 18 B5 73  ┌?µs↓"µs #µsâ↑µs
00504560  B4 8D B5 73 00 00 00 00 EC 14 0F 77 DD F7 0F 77  ┤ìµs....ÿ¶*w¦÷*w
00504570  FC 2C 0F 77 7E 36 0F 77 BD 37 0F 77 67 33 0F 77  ⁿ,*w~6*w¢7*wg3*w
00504580  8E F3 0F 77 5C CC 12 77 E4 F3 0F 77 B1 CC 12 77  Ä%*w\╠↨wõ%*w▒╠↨w
00504590  DC CC 12 77 8F 3D 0F 77 59 2F 14 77 74 16 0F 77  ▄╠↨wÅ=*wY/¶wt▬*w
005045A0  4B 17 0F 77 50 16 0F 77 AE A2 14 77 44 16 0F 77  K↕*wP▬*w«¢¶wD▬*w
005045B0  A4 F3 0F 77 00 00 00 00 A7 66 47 77 80 07 40 77  ñ%*w....§fGwÇ•@w
005045C0  E1 1C 40 77 9B 2B 47 77 0B 0D 40 77 97 28 40 77  ßL@w¢+Gw♂.@wù(@w
005045D0  56 B0 3F 77 3A DE 43 77 DF 06 3D 77 B9 29 40 77  V▒?w:ÏCw▀♠=w╣)@w
005045E0  BD AC 3F 77 8B 66 47 77 31 27 40 77 1D C5 3C 77  ¢¼?wïfGw1'@w#┼<w
005045F0  7C D5 3C 77 00 00 00 00 99 8A 2A 77 00 00 00 00  ¦'<w....öè*w....
00504600  B8 DC 16 00 D8 DC 16 00 F8 DC 16 00 18 DD 16 00  ©▄..ï▄..°▄..↑▌..
```

jooooooder (perdón, jeje…) ¿qué pasa? Si está igual o pero que antes..!! es evidente que algo nos está fallando y bastante, vamos a investigarlo, reiniciamos Olly y dejamos correr el programa, se detendrá en el HBP del primer mágico, los nopeamos como antes para ir bien, luego sabemos que retorna la API en EAX como ya vimos antes, aquí:

```
00880038   90              NOP
00880039   90              NOP
0088003A   3B35 08FB8900   CMP ESI,DWORD PTR DS:[89FB08]        USER32.77D10000
00880040   90              NOP
00880041   90              NOP
00880042   53              PUSH EBX
00880043   56              PUSH ESI
00880044   E8 5B6DF7FF     CALL VP.007F6DA4                     ;pone la API en EAX...
00880049   8BD8            MOV EBX,EAX
0088004B   EB 5A           JMP SHORT VP.008800A7
```

Traceamos con F8 para superar el CALL de la dirección :880044 y miramos los registros:

```
Registers (MMX)
EAX 73B51883 MSVFW32.DrawDibDraw
ECX 00F16FE0 ASCII "DrawDibDraw"
EDX 77FB306C ntdll.77FB306C
EBX 00F16FE0 ASCII "DrawDibDraw"
ESP 0012F524
EBP 0012F544
ESI 73B50000 MSVFW32.73B50000
EDI 00F12028
```

Pues parece que va todo perfecto ¿? Dejamos el HBP y seguimos mirando los valores de retorno, a nosotros en principio el que nos interesa es el que trae EAX que es el que tiene el valor correcto de la API, pero:

```
Registers (MMX)
EAX 0016B9E8
ECX 77E5AC72 kernel32.VirtualAlloc
EDX 00000001
EBX 00F16FFC ASCII "VirtualAlloc"
ESP 0012F524
EBP 0012F544
ESI 77E40000 kernel32.77E40000
EDI 00F12028
```

De repente comienza a retornar los valores buenos en ECX ¿? Es por esto que la IAT no se reparó bien, claro le estábamos pasndo el contenido de EAX y aquí se puede apreciar que es un valor del tipo 16B9E8 que no es ninguna API, así pues vemos que hay DLL's que trata de distinta manera, en concreto encontré que lo hacía con la kernel32, user32, y la ntdll, estas tres son las que nos dan problemas aunque bastante solucionables por cierto no encierra dificultad alguna sabiendo además en qué rango está cada API:

Kernel32 => 504268 a 504548
User32 => 504600 a 504944
Ntdll => No aparece ¿las tratará como si fueran del kernel?

Lo único que debemos es tener presente que cuando vaya a tratar una Api del kernel o de la User32 se le deberá pasar a EBX el contenido de ECX y no el de EAX com oveníamos haciendo, podríamos injertar, hacer un script, etc., para solucionar esto, osotros nos haremos un sencillo script para que nos ayude, jeje…

```
-----------/ inicio_script
inicio:
eob vamos
run

vamos:
CMP eip, 880038
jne no_cambio
MOV eip, 880042
jmp inicio

no_cambio:
CMP eip, 880049
JNE termina
CMP esi, 77e40000
JE cambio
CMP esi, 77d10000
JE cambio
JMP inicio



cambio:
MOV ebx, ecx
MOV eax, ecx
jmp inicio


termina:
ret
-----------/ final_script
```

Os explico en qué consiste ya que no es complicado y es muy simple de explicar: primero creamos la etiqueta "inicio" que será el inicio del script

```
inicio:
eob vamos
run
```

le sigue el "eob vamos" que pasa el control a la etiqueta "vamos" en caso de que ocurra una excepción o encuentre algún tipo de BP, eob significa Exception_Or_BreakPoint si no

estoy mal informado ya sabéis que no me gusta engañar a nadie aunque sea de forma involuntaria, sigue con el bloque "vamos" que es este:

```
vamos:
CMP eip, 880038
jne no_cambio
MOV eip, 880042
jmp inicio
```

Lo que hace es mirar si EIP=880038 que es la dirección del mágico, si es así cambia el valor del mismo por EIP==880042 simulando el nopeo de ambos saltos, si no estamos ahí no lo modifica y vamos a la etiqueta "no_cambio" donde se continúa verificando el valor de EIP para saber si estamos en la instrucción en la que debemos actuar ;-) es esta:

```
no_cambio:
CMP eip, 880049
JNE termina
CMP esi, 77e40000
JE cambio
CMP esi, 77d10000
JE cambio
JMP inicio
```

Sencilla explicación, si EIP=!880049 significa que el script debe terminar ya que el BP que ha saltado no es el que queremos, en cambio si estamos en :880049 continua y verifica el valor de ESI, concretamente mira si tiene la base del kernel o la del userque son las DLL's que nos torpedea, recordemos que a la ntdll también le mete mano pero las trata como si fueran del kernel por eso ya nos vale con esto, debemos también advertir que cuando estamos en la instrucción:

```
00880049    8BD8            MOV EBX,EAX
```

que es donde debemos cambiar los valores MALOS por los BUENOS tenemos en ESI la base de la DLL que está tratando, esto nos vale a nosotros para ver si está con el kernel o la user, de ahí que miremos el valor de ESI en el bloque anterior y actuemos en consecuencia en función del mismo ;-)

Después le llega el turno a la etiqueta "cambio" que es la contraria a "no_cambio" claro está, veamos en qué consiste:

```
cambio:
MOV ebx, ecx
MOV eax, ecx
jmp inicio
```

Nada sencillito también, lo que hace es poner el valor BUENO que está en ECX en EAX y EBX, en principio pensé que con ponerlo en EBX valdría pero no sé porqué si no lo ponemos también en EAX el script no chuta ¿? Después de cada bloque siempre viene un salto incondicional que nos lleva al inicio, esto es para que el script no finalice y termine el trabajo.

Por último lo más sencillo, jeje..

```
termina:
ret
```

Pues eso que retorna el control a Olly, como tenemos un HBP en el OEP nos detenemos allí, si miramos ahora el primer CALL a la IAT que antes era:

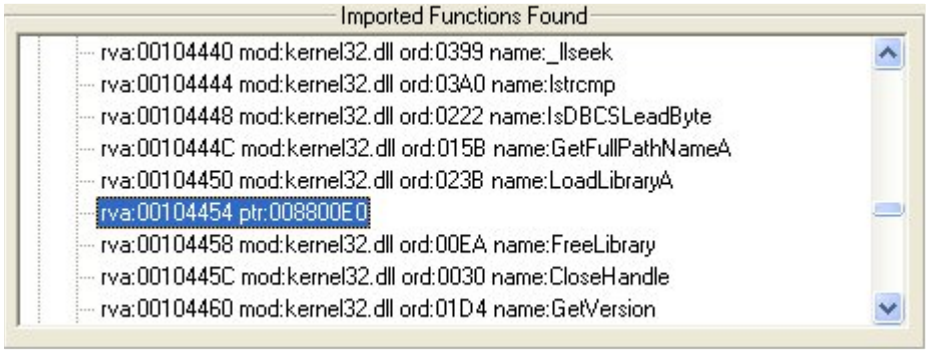
```
004A3291    FF15 60445000   CALL DWORD PTR DS:[504460]
```

Así tal cual sin ninguna API, ahora vemos que efectivamente apunta a GetVersion, jeje… el script ha hecho bien su trabajo parece ser, veamos a lo que me refiero:

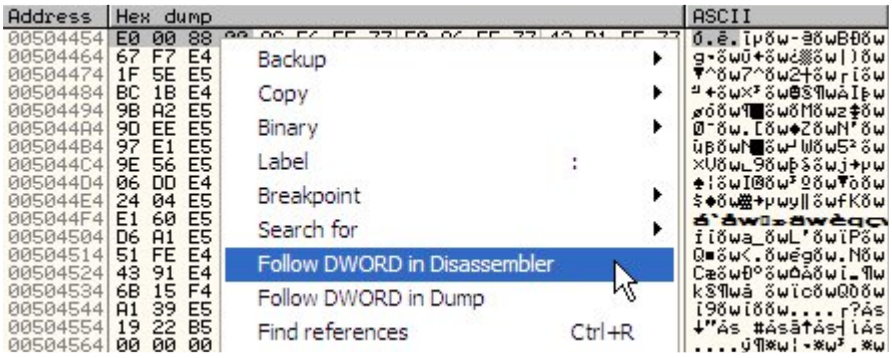
```
004A326B   55              PUSH EBP
004A326C   8BEC            MOV EBP,ESP
004A326E   6A FF           PUSH -1
004A3270   68 08FC5200     PUSH wbb.0052FC08
004A3275   68 78164A00     PUSH wbb.004A1678                    SE handler installation
004A327A   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
004A3280   50              PUSH EAX
004A3281   64:8925 000000  MOV DWORD PTR FS:[0],ESP
004A3288   83EC 58         SUB ESP,58
004A328B   53              PUSH EBX
004A328C   56              PUSH ESI
004A328D   57              PUSH EDI
004A328E   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
004A3291   FF15 60445000   CALL DWORD PTR DS:[504460]          kernel32.GetVersion
004A3297   33D2            XOR EDX,EDX
```

Vemos como ahora sí que apunta a la API cosa que antes no ocurría, este además es un buen momento para dumpear, no agregaré ninguna imagen ya que es un paso muy sencillo y está además muy bien explicado en muchos otros tutes, sólo decir que lo que debemos hacer el pulsar una vez en la opción "Get EIP as OEP" para que nos deje el OEP correcto y destildar la opción "Rebuild IAT…" el resto pulsando en Dump, le damos un nombre tute.exe por ejemplo y ya tenemos el archivo en el disco, ahora vamos a reparar la IAT, abrimos el ImpRec y le damos los valores correctos:

Los datos del campo OEP los obtenemos restando el imagebase 400000 al OEP 4A326B, así nos queda este A326B que vemos en la imagen, ocurre lo mismo para el valor RVA que es el inicio de la IAT (el offset también) y por último el Size que es el tamaño de ésta, cuando lo tengamos picamos en "Get Imports" y nos dice que quedó una entrada por resolver, bueno es sólo una, al lado de las más de 300 de antes no es nada ;-)



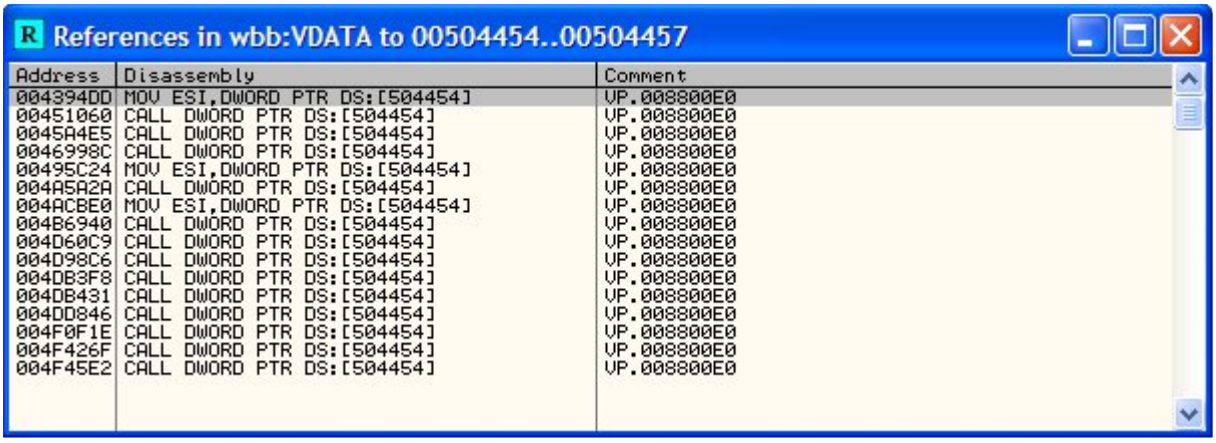
Esta es la que quedó por reparar, tiene muy pero que muy fácil solución: la primera es con el propio Import con la opción "Trace Level1 (Disasm)" que la saca rápido, y la otra es más manual y me gusta más, vemos que el offset de la entrada es 104454, si le sumamos la imagebase nos queda :504454 pues allí está la entrada MALA, vamos a esta dirección en la ventana de Dump (ventana de datos) en el Olly donde tenemos el programa detenido en el OEP con la IAT casi correcta y le seguimos la pista así:



cuando picamos con el ratón en esta opción "Follow DWORD in Disassembler" le estamos diciendo a Olly que queremos ver el código de la dirección a la que apunta la entrada, en este caso :008800E0 y que además lo haga en la ventana de código ya que queremos verlo bien, de esta manera nos aparece el código de la entrada MALA:

```
008800E0   55              PUSH EBP
008800E1   8BEC            MOV EBP,ESP
008800E3   83C4 F8         ADD ESP,-8
008800E6   53              PUSH EBX
008800E7   56              PUSH ESI
008800E8   33C0            XOR EAX,EAX
008800EA   8945 F8         MOV DWORD PTR SS:[EBP-8],EAX
008800ED   8B5D 0C         MOV EBX,DWORD PTR SS:[EBP+C]
008800F0   8B75 08         MOV ESI,DWORD PTR SS:[EBP+8]
008800F3   33C0            XOR EAX,EAX
008800F5   55              PUSH EBP
008800F6   68 89018800     PUSH VP.00880189
008800FB   64:FF30         PUSH DWORD PTR FS:[EAX]
008800FE   64:8920         MOV DWORD PTR FS:[EAX],ESP
00880101   8BC3            MOV EAX,EBX
00880103   E8 D076F7FF     CALL VP.007F77D8
00880108   66:85C0         TEST AX,AX
0088010B   75 2A           JNZ SHORT VP.00880137
0088010D   3B35 04FB8900   CMP ESI,DWORD PTR DS:[89FB04]        kernel32.77E40000
00880113   75 0E           JNZ SHORT VP.00880123
00880115   66:81FB 5401    CMP BX,154
0088011A   75 07           JNZ SHORT VP.00880123
0088011C   BB E0008800     MOV EBX,VP.008800E0
00880121   EB 50           JMP SHORT VP.00880173
00880123   55              PUSH EBP
00880124   8BCB            MOV ECX,EBX
00880126   8BD6            MOV EDX,ESI
00880128   A1 14FB8900     MOV EAX,DWORD PTR DS:[89FB14]
0088012D   E8 E2FEFFFF     CALL VP.00880014
00880132   59              POP ECX
00880133   8BD8            MOV EBX,EAX
00880135   EB 3C           JMP SHORT VP.00880173
00880137   8D45 F8         LEA EAX,DWORD PTR SS:[EBP-8]
0088013A   8BD3            MOV EDX,EBX
0088013C   E8 4746F7FF     CALL VP.007F4788
00880141   8B45 F8         MOV EAX,DWORD PTR SS:[EBP-8]
00880144   BA A4018800     MOV EDX,VP.008801A4                  ASCII "GetProcAddress"
00880149   E8 4289F7FF     CALL VP.007F8A90
0088014E   84C0            TEST AL,AL
00880150   74 0F           JE SHORT VP.00880161
00880152   3B35 04FB8900   CMP ESI,DWORD PTR DS:[89FB04]        kernel32.77E40000
00880158   75 07           JNZ SHORT VP.00880161
```

Y allí mismo vemos referencias directas a la propia API, otra solución sería esta: buscamos referencias a la entrada MALA con Ctrl+R y nos aparece esto:



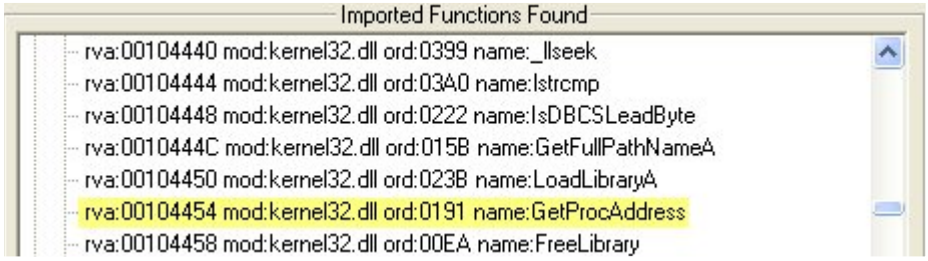
References in wbb:VDATA to 00504454..00504457

Address	Disassembly	Comment
004394DD	MOV ESI,DWORD PTR DS:[504454]	VP.008800E0
00451060	CALL DWORD PTR DS:[504454]	VP.008800E0
0045A4E5	CALL DWORD PTR DS:[504454]	VP.008800E0
0046998C	CALL DWORD PTR DS:[504454]	VP.008800E0
00495C24	MOV ESI,DWORD PTR DS:[504454]	VP.008800E0
004A5A2A	CALL DWORD PTR DS:[504454]	VP.008800E0
004ACBE0	MOV ESI,DWORD PTR DS:[504454]	VP.008800E0
004B6940	CALL DWORD PTR DS:[504454]	VP.008800E0
004D60C9	CALL DWORD PTR DS:[504454]	VP.008800E0
004D98C6	CALL DWORD PTR DS:[504454]	VP.008800E0
004DB3F8	CALL DWORD PTR DS:[504454]	VP.008800E0
004DB431	CALL DWORD PTR DS:[504454]	VP.008800E0
004DD846	CALL DWORD PTR DS:[504454]	VP.008800E0
004F0F1E	CALL DWORD PTR DS:[504454]	VP.008800E0
004F426F	CALL DWORD PTR DS:[504454]	VP.008800E0
004F45E2	CALL DWORD PTR DS:[504454]	VP.008800E0

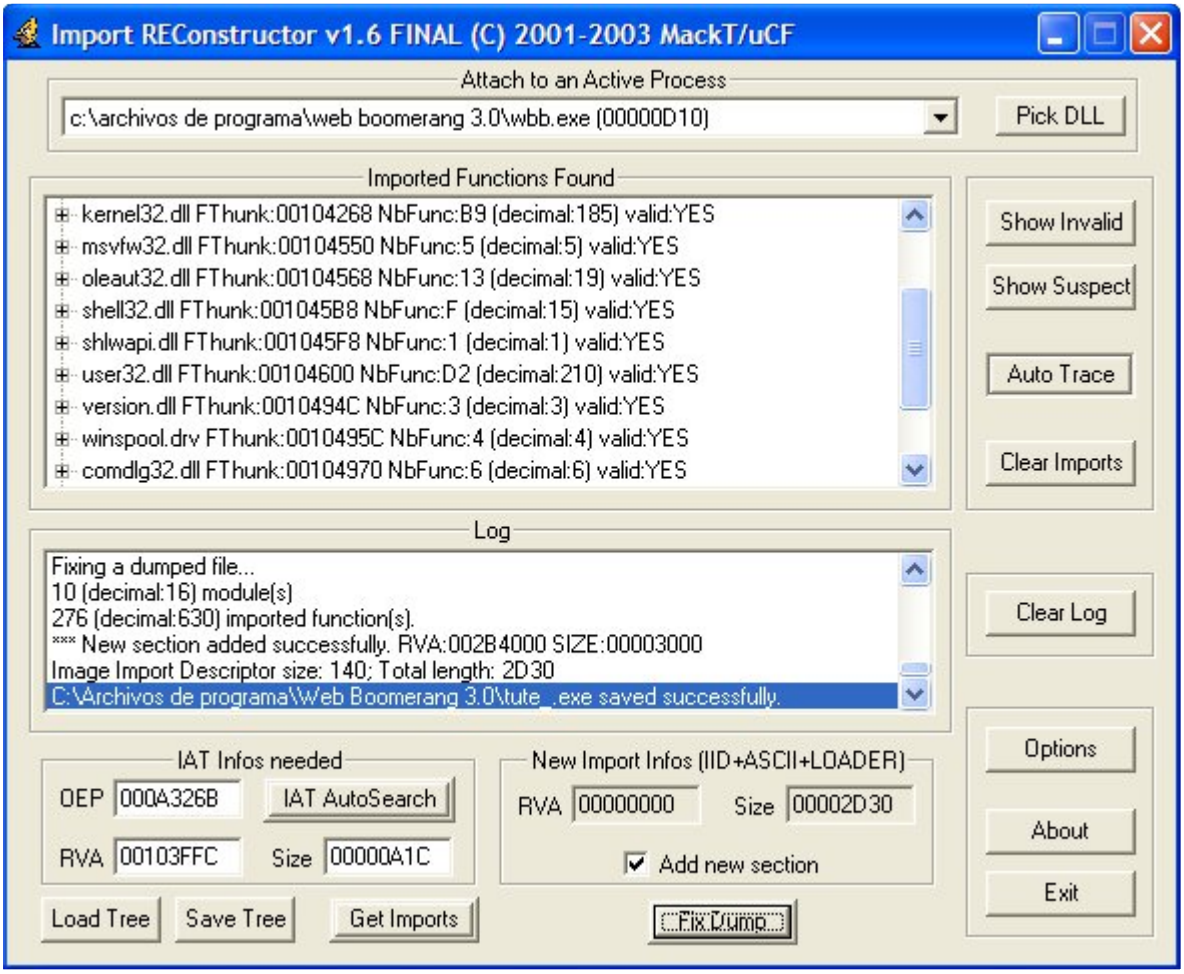
Elejimos por ejemplo la de :451060 y pulsamos Enter, vemos el código:

```
0045103E   C705 A8505400   MOV DWORD PTR DS:[5450A8],1
00451048   68 EC565400     PUSH wbb.005456EC                    ASCII "shell32.dll"
0045104D   FF15 50445000   CALL DWORD PTR DS:[504450]           kernel32.LoadLibraryA
00451053   3BC3            CMP EAX,EBX
00451055   8945 F8         MOV DWORD PTR SS:[EBP-8],EAX
00451058   74 73           JE SHORT wbb.004510CD
0045105A   68 DC565400     PUSH wbb.005456DC                    ASCII "SHGetSettings"
0045105F   50              PUSH EAX
00451060   FF15 54445000   CALL DWORD PTR DS:[504454]           VP.008800E0
00451066   3BC3            CMP EAX,EBX
00451068   74 5A           JE SHORT wbb.004510C4
0045106A   56              PUSH ESI
```

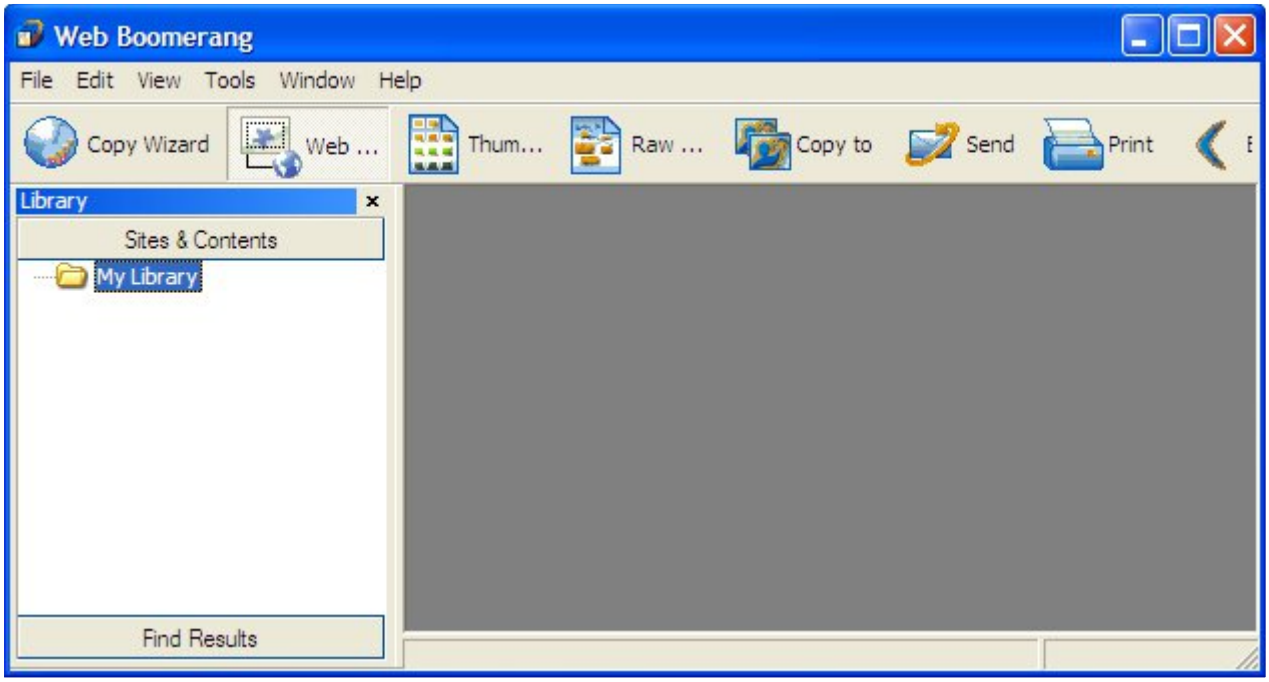
Resalto esos dos PUSH porque sabemos que la API GetProcAddress recibe dos parámetros: la base de la DLL y el nombre de la API, por ejemplo si queremos saber la dirección de GetVersion los parámetros serían 77E40000 que es la base del kernel y la string "GetVersion" y nos retornará en EAX la dirección de esta API, si ponemos un BP en :451060 veremos que efectivamente se trata de GetProcAddress así que la reparamos ya:



Una vez que lo tenemos todo pulsamos "Fix Dump" seleccionando el ejecutable tute.exe que antes dumpeamos, nos creará el archivo tute_.exe como se puede leer aquí:



Este tute_.exe es el archivo que debemos probar, le hacemos doble clic y…



jeje… lo hemos conseguido, el archivo funciona a la perfección ;-)

Otro detalle a tener en cuenta es que ya no expirará más debido a que es el propio packer quién se encargaba de esto, en concreto mediante la librería VP.dll que tiene en su misma carpeta, esta es la que se encargaba de todo, nuestro dumpeado ya no la necesita y ni si quiera la tiene en su IAT así queni la importa, podemos eliminarla de la carpeta y seguirá funcionando, y como digo lo mejor sin límite de tiempo ;-)

Bueno aquí termina este tute, espero que os haya entretenido un rato, he de decir que esta protección es un tanto antigua y no cuenta con ningún método antidebug, ni rompe la cabecera (PE header), etc., como hacen la mayoría de nuevos packers, aún así he creído conveniente escribir sobre él nunca está de más pienso yo…

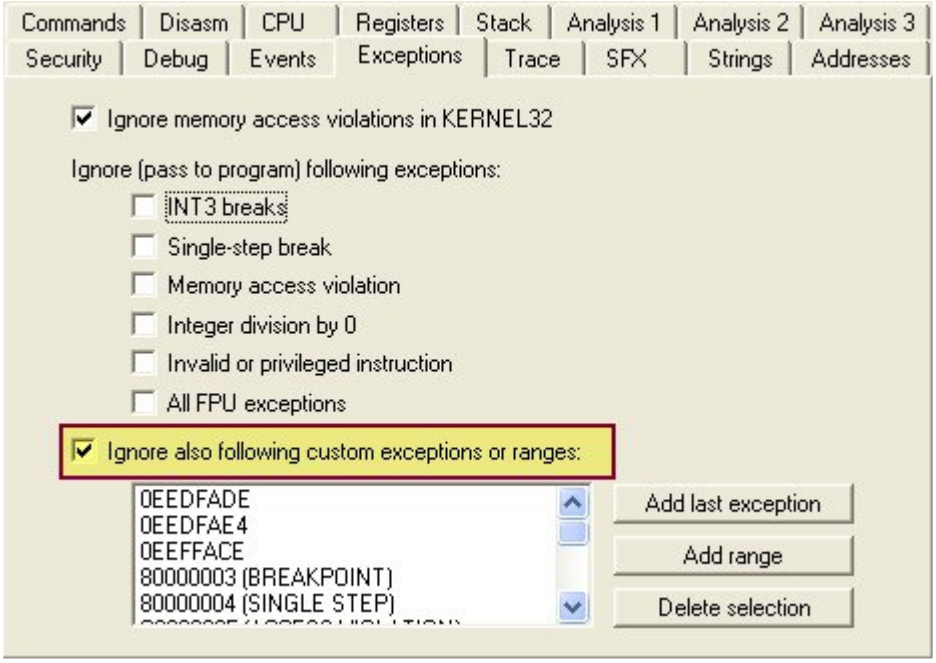
Ahora que hemos terminado podemos poner la guinda al pastel desempakando la otra aplicación que trae también con Visual Protect, el ejecutable se llama "wbmerang.exe" y está en la misma carpeta de instalación.

Vamos a por el amigo, jeje… Si lo miramos con PEiD nos dice:

Visual Protect -> Visage

O sea que lo mismo que antes, en este caso lo desempakaremos sin más no vamos a volver a explicar todo el rollo otra vez, jeje.. sería de locos, iremos viendo lo esencial ya que leyendo la primera parte del tute podemos hacerlo sin problemas.

Abrimos un Olly y cargamos "wbmerang.exe" se detiene en el EP después de avisarnos de que tanto este ejecutable como la librería vp.dll tienen el EP fuera de la sección de código, primer síntoma de empaketamiento ;xD de momento vamos a por el OEP así de entrada, el método que yo utilizo es muy sencillo, desactivo las excepciones:



Seleciono la opción "Ignore also following…" para que Olly no se detenga en las excepciones que tengamos añadidas, en este caso yo tengo muchas si alguna no la tuviera la añado con "Add last exception" y queda en la lista, una vez configurado así pulso F9 y me aparece la nag de VisualProtect que antes vimos, es la misma puesto que se encuentra en vp.dll y esta librería es llamada desde ambos ejecutables (el que antes desempakamos y con el que estamos ahora) mientras está la nag en pantalla y sin Aceptar voy al Memory Map (ventana de memoria) con Alt+M y pongo un BPM en la primera sección que ya la sabemos encontrar, después sí acepto la nag y Olly para por el BPM:

```
0046E004  F2:AE        REPNE SCAS BYTE PTR ES:[EDI]
0046E006  F7D9         NEG ECX
0046E008  03CB         ADD ECX,EBX
0046E00A  8BFE         MOV EDI,ESI
0046E00C  8B75 0C      MOV ESI,DWORD PTR SS:[EBP+C]
```

Pero no está ejecutando código como vemos en la barra de estado:

Está leyendo de :401000 que es el inicio de la primera sección precisamente, como no nos interesa seguimos, pero si vamos pulsando F9 cada vez va parando en el mismo sitio y va leyendo de :401001, :401002, etc., y nos podemos pasar una tarde con F9, para evitarlo vamos a la ventana de memoria otra vez con Alt+M y quitamos el BPM igual que lo pusimos pero eligiendo la opción "Remove memory breakpoint" y nos vamos a la ventana de código con Alt+C a partir de ahora no tenemos el BPM cuidado en pulsar F9 que el programa correrá, lo que haremos será dehacer el bucle en el que estamos metidos, traceamos hasta el RET que lo pasamos y salimos a la zona del bucle maldito:

```
0046D9C2  8B15 EC2E4700  MOV EDX,DWORD PTR DS:[472EEC]   wbmerang.00400000
0046D9C8  0351 0C        ADD EDX,DWORD PTR DS:[ECX+C]
0046D9CB  52             PUSH EDX
0046D9CC  E8 1F060000    CALL wbmerang.0046DFF0
0046D9D1  83C4 0C        ADD ESP,0C
0046D9D4  85C0           TEST EAX,EAX
0046D9D6  75 0C          JNZ SHORT wbmerang.0046D9E4
0046D9D8  8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
0046D9DB  50             PUSH EAX
0046D9DC  E8 D9E6FFFF    CALL wbmerang.0046C0BA
0046D9E1  83C4 04        ADD ESP,4
0046D9E4  8B4D FC        MOV ECX,DWORD PTR SS:[EBP-4]
0046D9E7  83C1 28        ADD ECX,28
0046D9EA  894D FC        MOV DWORD PTR SS:[EBP-4],ECX
0046D9ED  EB 9C          JMP SHORT wbmerang.0046D98B
0046D9EF  5E             POP ESI
0046D9F0  8BE5           MOV ESP,EBP
0046D9F2  5D             POP EBP
0046D9F3  C3             RETN
```

Ese JMP nos envía para arriba pues nada un BP debajo justo de él y damos RUN, cuando Olly pare quitamos el BP y superamos el RET, de esta manera caemos cerca del salto al OEP que sabemos que lo hace mediante la instrucción "JMP DWORD PTR [ESP-4]" y es este:

```
00675FC7   B8 08000000    MOV EAX,8
00675FCC   E8 AF2FF6FF    CALL VP.005D8F80
00675FD1   8B8D 58FAFFFF  MOV ECX,DWORD PTR SS:[EBP-5A8]
00675FD7   8D85 5CFAFFFF  LEA EAX,DWORD PTR SS:[EBP-5A4]
00675FDD   BA 946B6700    MOV EDX,VP.00676B94                ASCII "Finalizing 0x"
00675FE2   E8 B5E8F5FF    CALL VP.005D489C
00675FE7   8B85 5CFAFFFF  MOV EAX,DWORD PTR SS:[EBP-5A4]
00675FED   E8 46BCFFFF    CALL VP.00671C38
00675FF2 - FF65 FC        JMP DWORD PTR SS:[EBP-4]           wbmerang.00416F5B
00675FF5   6A 00          PUSH 0
```

Vemos una referencia a "Finalizing 0x" como veíamos en el desempakado anterior, esto significa que vamos bien, ese salto nos pone en el OEP, jeje…

```
00416F5B   55             PUSH EBP
00416F5C   8BEC           MOV EBP,ESP
00416F5E   6A FF          PUSH -1
00416F60   68 D02E4400    PUSH wbmerang.00442ED0
00416F65   68 98AA4100    PUSH wbmerang.0041AA98
00416F6A   64:A1 00000000 MOV EAX,DWORD PTR FS:[0]
00416F70   50             PUSH EAX
00416F71   64:8925 000000 MOV DWORD PTR FS:[0],ESP
00416F78   83EC 58        SUB ESP,58
00416F7B   53             PUSH EBX
00416F7C   56             PUSH ESI
00416F7D   57             PUSH EDI
00416F7E   8965 E8        MOV DWORD PTR SS:[EBP-18],ESP
00416F81   FF15 94E24300  CALL DWORD PTR DS:[43E294]   <-- Habrá que reparar esto ¡!
00416F87   33D2           XOR EDX,EDX
```

Igual que antes vemos un CALL que apunta no se sabe donde, a una api seguro que no porque no sale su nombre al lado, como pongo en la imagen habrá que reparar esto, de momento buscaremos el inicio y final de tabla, haciendo un "Follow in dump->Memory address" en el CALL que vemos en la imagen caemos en plena tabla, desde aquí subimos un poco y encontramos el inicio de la misma:

```
Address   Hex dump                                          ASCII
0043DFE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0043DFF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0043E000  10 24 DA 77 D9 23 DA 77 D4 65 DB 77 9A 22 DA 77  ►$┌w┘#┌wÊe╠wÜ"┌w
0043E010  B1 63 DB 77 BB 28 DA 77 D6 27 DA 77 E2 68 DB 77  ▒c╠w╗(┌w╓'┌w┌h╠w
0043E020  5B 66 DB 77 69 6D DC 77 8E 5B DC 77 4D 22 DB 77  [f╠wim▄wÄ[▄wM"╠w
0043E030  D8 17 DA 77 00 00 00 00 25 74 32 77 46 EE 34 77  ï↨┌w....%t2wF^4w
```

Lo anotamos y buscamos también el final:

```
Address   Hex dump                                          ASCII
0043E5C0  34 F1 BF 00 58 F1 BF 00 80 F1 BF 00 A4 F1 BF 00  4±┐.X±┐.Ç±┐.ñ±┐.
0043E5D0  C8 F1 BF 00 EC F1 BF 00 14 F2 BF 00 3C F2 BF 00  ╚±┐.ý±┐.¶≡┐.<≡┐.
0043E5E0  00 00 00 00 CE 31 1B 76 7D 01 1C 76 4B 97 1B 76  ....╬1←v}0└vKù←v
0043E5F0  3C FA 1B 76 E9 92 1B 76 89 06 1C 76 0A 6C 1B 76  <·←vúÉ←vë♠└v.l←v
0043E600  E6 5D 1B 76 C6 4D 1D 76 17 C2 1B 76 CF 79 1B 76  µ]←vãM↔v↨┬←vÏy←v
0043E610  00 00 00 00 63 16 F8 72 AD 5F F8 72 69 5E F8 72  ....c▬°r↓_°ri^°r
0043E620  00 00 00 00 CF 90 36 76 90 23 36 76 88 1B 36 76  ....ÏÉ6vÉ#6vê←6v
0043E630  00 00 00 00 C3 EA 4C 00 77 21 4B 00 58 ED 4C 00  ....├ÊL.w!K.XíL.
0043E640  00 00 00 00 00 00 00 00 10 FE 43 00 50 E6 43 00  ........►■C.PµC.
0043E650  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lo anotamos también, en la imagen además vemos referencias a zonas del tipo :BFFxxx que son zonas que nuestro dumpeado no tendrá, necesitamos reparar esto como hicimos con el ejecutable anterior, elegimos una entrada MALA, por ejemplo [43E104]==BFF9A0 y le ponemos un HBPWD, reiniciamos Olly, damos RUN y esperamos a que escriba los valores finales, esto lo hace aquí:

```
00660449   8B40 E8        MOV EAX,DWORD PTR DS:[EAX-18]
0066044C   E8 63FDFFFF   CALL VP.006601B4                   ;call GetProcAddress
00660451   8BD8           MOV EBX,EAX
00660453   8B07           MOV EAX,DWORD PTR DS:[EDI]
00660455   8918           MOV DWORD PTR DS:[EAX],EBX        ;aqui guarda...
00660457   85DB           TEST EBX,EBX
00660459 v 75 0E          JNZ SHORT VP.00660469
0066045B   8B45 08        MOV EAX,DWORD PTR SS:[EBP+8]
0066045E   8B40 E4        MOV EAX,DWORD PTR DS:[EAX-1C]
00660461   83C0 02        ADD EAX,2
00660464   E8 EBFDFFFF   CALL VP.00660254
00660469   8306 04        ADD DWORD PTR DS:[ESI],4
0066046C   8307 04        ADD DWORD PTR DS:[EDI],4
0066046F   8B06           MOV EAX,DWORD PTR DS:[ESI]
00660471   8B18           MOV EBX,DWORD PTR DS:[EAX]
00660473   85DB           TEST EBX,EBX
00660475 ^ 0F85 61FFFFFF  JNZ VP.006603DC                   ;fin loop...
0066047B   5F             POP EDI
0066047C   5E             POP ESI
0066047D   5B             POP EBX
0066047E   5D             POP EBP
0066047F   C3             RETN
```

rellena la entrada en :660455 como vemos en la imagen, el CALL anterior pone la API en EAX así que el mágico está dentro de este CALL lo tendremos en cuenta, después más abajo vemos un JNZ que nos envía al inicio de la rutina en la que estamos. Es más es una zona muy similar a la del exe que estudiamos anteriormente, eso sí con algunas diferencias que veremos rápidamente, el call GetProcAddress lo encontramos en :66044C entramos en él y buscamos el subcall que pone la API correcta en EAX, está en :660205 le ponemos un BP y entramos, seguimos buscando hasta localizar la zona clave, he de decir que encontré algo interesante respecto a la API GetProcAddress, veamos la zona:

```
0066013C   E8 4746F7FF   CALL VP.005D4788
00660141   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
00660144   BA A4016600   MOV EDX,VP.006601A4          ASCII "GetProcAddress"
00660149   E8 4289F7FF   CALL VP.005D8A90             ;mira si es GetProcAddress ??
0066014E   84C0          TEST AL,AL
00660150 v 74 0F         JE SHORT VP.00660161
00660152   3B35 04FB6700 CMP ESI,DWORD PTR DS:[67FB04] kernel32.77E40000
00660158 v 75 07         JNZ SHORT VP.00660161
0066015A   BB E0006600   MOV EBX,VP.006600E0          ;ojo a este valor...
0066015F v EB 12         JMP SHORT VP.00660173
00660161   55            PUSH EBP
00660162   8BCB          MOV ECX,EBX
00660164   8BD6          MOV EDX,ESI
00660166   A1 14FB6700   MOV EAX,DWORD PTR DS:[67FB14]
0066016B   E8 A4FEFFFF   CALL VP.00660014             ;call GetProcAddress
00660170   59            POP ECX
00660171   8BD8          MOV EBX,EAX
00660173   33C0          XOR EAX,EAX
```

Lo que quiero mostraros es el trato especial que le da a GetProcAddress, hace lo siguiente: sabemos que estamos en un loop donde se van tratando todas las entradas de la IAT, perfecto, en unas no se entromete como COMCTL, GDI, VERSION.dll, etc., y en otras sí como es el caso del kernel32.dll, user32.dll pero hay más, dentro del propio kernel hace una diferenciación clara, va mirando de que api se trata hasta que se da cuenta que es GetProcAddress, entonces mete el valor 6600E0 en EBX que será finalmente el que escriba en la IAT, es decir que redirigirá la API a esta zona, pero esto sólo lo hace con esta ¿? lo tendremos en cuenta para el script, veamos lo que quiero decir:

Estamos en :660149 como marca la imagen anterior, miremos los registros:

```
Registers (FPU)
EAX 00BF6C30 ASCII "ExitThread"
ECX 00000000
EDX 006601A4 ASCII "GetProcAddress"
EBX 00BF4B08 ASCII "ExitThread"
ESP 0012F550
EBP 0012F56C
ESI 77E40000 kernel32.77E40000
EDI 0012F5D4
```

La API que está tratando ahora es ExitThread, se mete en el CALL de :660149 y mira si es GetProcAddress, se da cuenta de que no es:

```
0066013C   E8 4746F7FF   CALL VP.005D4788
00660141   8B45 F8       MOV EAX,DWORD PTR SS:[EBP-8]
00660144   BA A4016600   MOV EDX,VP.006601A4          ASCII "GetProcAddress"
00660149   E8 4289F7FF   CALL VP.005D8A90             ;mira si es GetProcAddress ??
0066014E   84C0          TEST AL,AL
00660150 v 74 0F         JE SHORT VP.00660161
00660152   3B35 04FB6700 CMP ESI,DWORD PTR DS:[67FB04] kernel32.77E40000
00660158 v 75 07         JNZ SHORT VP.00660161
0066015A   BB E0006600   MOV EBX,VP.006600E0          ;ojo a este valor...
0066015F v EB 12         JMP SHORT VP.00660173
00660161   55            PUSH EBP
00660162   8BCB          MOV ECX,EBX
00660164   8BD6          MOV EDX,ESI
00660166   A1 14FB6700   MOV EAX,DWORD PTR DS:[67FB14]
0066016B   E8 A4FEFFFF   CALL VP.00660014             ;call GetProcAddress
00660170   59            POP ECX
```

Y salta evitando meter el valor MALO en EBX (me refiero a ese 6600E0) lo que haremos será poner un BP en 660152 y quitar los que tengamos (excepto el del OEP claro) y damos RUN, cuando pare querrá decir que ha encontrado GetProcAddress ya que no se ejecutó el salto, así nos evitamos pulsar F9 cada vez y vamos directos al grano, en la siguiente instrucción:

```
00660152    3B35 04FB6700    CMP ESI,DWORD PTR DS:[67FB04]      ; kernel32.77E40000
```

se asegura de estar tratando la API adecuada pues mira si es del kernel, GetProcAddress por supuesto que es del kernel así no habrá errores, es un paso tonto porque no hay otra GetProcAddress en otra DLL pero bueno ¿? cosas del programador… como ve que es la que busca nos mete ese valor 6600E0 en EBX y salta por encima del CALL que pone la API correcta en EAX, veámoslo:

```
00660144   BA A4016600   MOV EDX,VP.006601A4          ASCII "GetProcAddress"
00660149   E8 4289F7FF   CALL VP.005D8A90             ;mira si es GetProcAddress ??
0066014E   84C0          TEST AL,AL
00660150 v 74 0F         JE SHORT VP.00660161
00660152   3B35 04FB6700 CMP ESI,DWORD PTR DS:[67FB04] kernel32.77E40000
00660158 v 75 07         JNZ SHORT VP.00660161
0066015A   BB E0006600   MOV EBX,VP.006600E0          ;ojo a este valor...
0066015F v EB 12         JMP SHORT VP.00660173
00660161   55            PUSH EBP
00660162   8BCB          MOV ECX,EBX
00660164   8BD6          MOV EDX,ESI
00660166   A1 14FB6700   MOV EAX,DWORD PTR DS:[67FB14]
0066016B   E8 A4FEFFFF   CALL VP.00660014             ;call GetProcAddress
00660170   59            POP ECX
00660171   8BD8          MOV EBX,EAX
00660173   33C0          XOR EAX,EAX
```

Como se ve evitamos pasar por :66016B y para colmo de males este 6600E0 será el valor de retorno de la función (que lo llevará EAX) y lo escribirá en la IAT:

```
00660443   8D50 02      LEA EDX,DWORD PTR DS:[EAX+2]
00660446   8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
00660449   8B40 E8      MOV EAX,DWORD PTR DS:[EAX-18]
0066044C   E8 63FDFFFF  CALL VP.006601B4                ;call GetProcAddress
00660451   8BD8         MOV EBX,EAX
00660453   8B07         MOV EAX,DWORD PTR DS:[EDI]
00660455   8918         MOV DWORD PTR DS:[EAX],EBX      VP.006600E0
00660457   85DB         TEST EBX,EBX
00660459 v 75 0E        JNZ SHORT VP.00660469
0066045B   8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
```

Si miramos en la IAT efectivamente nos ha metido el GOL:

```
Address   Hex dump                                          ASCII
0043E2A8  34 12 C0 00 40 12 C0 00 28 C8 BF 00 50 C8 BF 00  4↕└.@↕└.(╚┐.P╚┐.
0043E2B8  5C C8 BF 00 5C 6D BF 00 3C C1 BF 00 94 C3 BF 00  \╚┐.\m┐.<┴┐.ö├┐.
0043E2C8  E0 00 66 00 74 9D 04 00 64 9D 04 00 52 9D 04 00  α.f.t♥♦.d♥♦.R♥♦.
0043E2D8  40 9D 04 00 30 9D 04 00 1E 9D 04 00 0E 9D 04 00  @♥♦.0♥♦.▲♥♦.♫♥♦.
0043E2E8  F8 9C 04 00 EC 9C 04 00 E4 9C 04 00 D6 9C 04 00  °£♦.∞£♦.ä£♦.í£♦.
```

Allí está la entrada [43E2C8] apuntará a 6600E0 en lugar de hacerlo a GetProcAddress como sabemos que debería hacer, si recordamos en el desempakado anterior nos quedó esta API sin reparar en el script, esto es porque se me pasó por alto este detalle, no importa añadiremos un par de líneas al mismo para que repare esta entrada también ;-)

Ahora vamos a encontrar el mágico que aún no lo hemos hecho, entramos en este CALL:

```
0066016B    E8 A4FEFFFF     CALL VP.00660014                    ; ;call GetProcAddress
```

Y allí encontramos algo conocido por nosotros:

```
0066002C   64:FF30      PUSH DWORD PTR FS:[EAX]
0066002F   64:8920      MOV DWORD PTR FS:[EAX],ESP
00660032   3B35 04FB6700 CMP ESI,DWORD PTR DS:[67FB04]   kernel32.77E40000
00660038 v 74 13        JE SHORT VP.0066004D             ;salto MALO...
0066003A   3B35 08FB6700 CMP ESI,DWORD PTR DS:[67FB08]   USER32.77D10000
00660040 v 74 0B        JE SHORT VP.0066004D             ;salto MALO...
00660042   53           PUSH EBX
00660043   56           PUSH ESI
00660044   E8 5B6DF7FF  CALL VP.005D6DA4                 ;pone la API en EAX...
00660049   8BD8         MOV EBX,EAX
0066004B v EB 5A        JMP SHORT VP.006600A7
0066004D   8BC3         MOV EAX,EBX
```

Estos son los dos saltos a evitar, necesitamos llegar siempre al CALL que hay en :660044 para llevarnos la API correcta, lo que no sabemos es si necesitaremos hacer el cambio de registros como en el scritp anteriro para que guarde los valores correctos en las DLL's kernel y user ya lo veremos, pues sí hay que hacerlo:

```
Registers (MMX)
EAX 0018EA10
ECX 77E4FD35 kernel32.WinExec
EDX 00000001
EBX 00BFCD90 ASCII "WinExec"
ESP 0012F524
EBP 0012F544
ESI 77E40000 kernel32.77E40000
EDI 00BF2030
```

cambié el FLAG_ZERO para que no saltara en :660038 cuando trataba una API del kernel y el resultado lo devuelve en ECX y no en EAX, o sea que haremos lo mismo que en el script de antes ya que nos funcionó bien, en conjunto será el mismo con las salvedades de que las direcciones lógicamente cambian y la entrada GetProcAddress que la trata de distinta manera como hemos visto, un primer intento podría ser este:

```
-----------/ inicio_script
inicio:
eob vamos
run

vamos:
CMP eip, 660152
Je Es_GetProcAddress
CMP eip, 660038
jne no_cambio
MOV eip, 8660042
jmp inicio

no_cambio:
CMP eip, 660049
```

```
JNE termina
CMP esi, 77e40000
JE cambio
CMP esi, 77d10000
JE cambio
JMP inicio

Es_GetProcAddress:
MOV eip, 660161
JMP inicio

cambio:
MOV ebx, ecx
MOV eax, ecx
jmp inicio


termina:
ret
-----------/ final_script
```

Cuando el script termina estamos en el OEP pero tenemos un problema, mirad la entrada perteneciente a GetProcAddress:

```
Address   Hex dump                                          ASCII
0043E2A8  40 E0 E4 77 C3 FA E4 77 02 15 F4 77 A0 FF E4 77  @ŏŏw├·ŏw☻§¶wá ŏw
0043E2B8  8B 63 E5 77 51 E3 E5 77 58 E3 E5 77 86 AD E5 77  ïcŏwQDŏwXDŏw♀↓ŏw
0043E2C8  80 64 69 00 35 FD E4 77 68 5A E5 77 79 BA E4 77  Çdi.5²ŏwhZŏwy‖ŏw
0043E2D8  8D BB E4 77 67 F7 E4 77 0D A1 E5 77 D6 A1 E5 77  ìŋŏwg-ŏw.íŏwííŏw
0043E2E8  A9 AD E5 77 4E 27 E5 77 EA 1B E4 77 9E 56 E5 77  @↓ŏwN'ŏwū←ŏwXVŏw
0043E2F8  24 04 E5 77 1C 39 E5 77 D9 57 E5 77 EC 5D E5 77  $◆ŏw∟9ŏw┘Wŏwý]ŏw
```

sigue estando mal ¿? Estudié el tema y me di cuenta que el script funciona pero la llamada a GetProcAddress retorna una dirección correcta pero dentro deo VP.dll esto se sabe haciendo un GO a la dirección que marca la entrada 696480 y vemos:

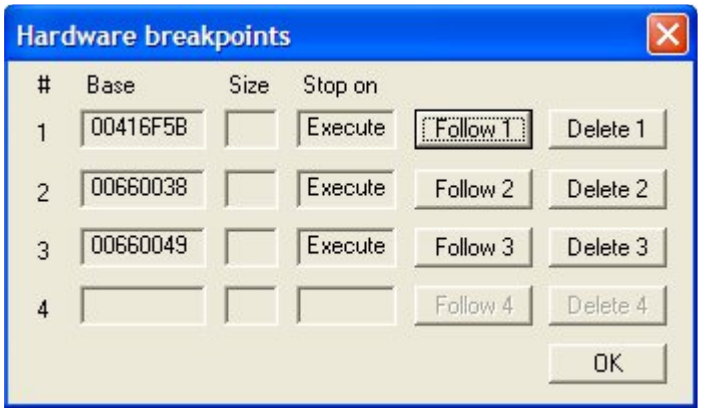
```
00692AEF   56              PUSH ESI
00692AF0   8B7424 0C       MOV ESI,DWORD PTR SS:[ESP+C]
00692AF4   8BC6            MOV EAX,ESI
00692AF6   C1E8 10         SHR EAX,10
00692AF9   66:85C0         TEST AX,AX
00692AFC ~ 75 1D           JNZ SHORT VP.00692B1B
00692AFE   8B4424 08       MOV EAX,DWORD PTR SS:[ESP+8]
00692B02   3905 A47D6900   CMP DWORD PTR DS:[697DA4],EAX
00692B08 ~ 75 07           JNZ SHORT VP.00692B11
00692B0A   66:81FE 5401    CMP SI,154
00692B0F ~ 74 27           JE SHORT VP.00692B38
00692B11   56              PUSH ESI
00692B12   50              PUSH EAX
00692B13   FF15 04406900   CALL DWORD PTR DS:[<&KERNEL32.Get kernel32.GetProcAddress
00692B19 ~ EB 2D           JMP SHORT VP.00692B48
00692B1B   68 80646900     PUSH VP.00696480                 ASCII "GetProcAddress"
00692B20   56              PUSH ESI
```

La dirección va variando entre ejecución y ejecución por eso no coincide pero es así como os lo explico que es tal como me pasó, entonces otra posibilidad es retocar el script un poco y otra la más sencilla es dejar que nos ponga la dirección que le de la gana y desde el ImpRec cambiarla, es lo que haremos, el script será este:

```
-----------/ inicio_script
inicio:
eob vamos
run

vamos:
CMP eip, 660038
jne no_cambio
MOV eip, 660042
jmp inicio

no_cambio:
CMP eip, 660049
JNE termina
CMP esi, 77e40000
JE cambio
CMP esi, 77d10000
JE cambio
JMP inicio
```

```
cambio:
MOV ebx, ecx
MOV eax, ecx
jmp inicio


termina:
ret
-----------/ final_script
```

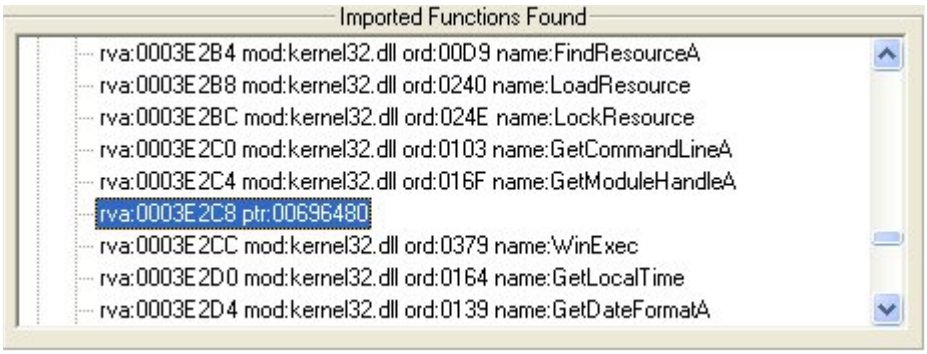
lo único que varía respecto al script utilizado en el otro exe son las direcciones, en lugar de 880038 será 660038, como vemos el offset sí que coincide, para ejecutar el script debemos tener puestos estos HBP's:



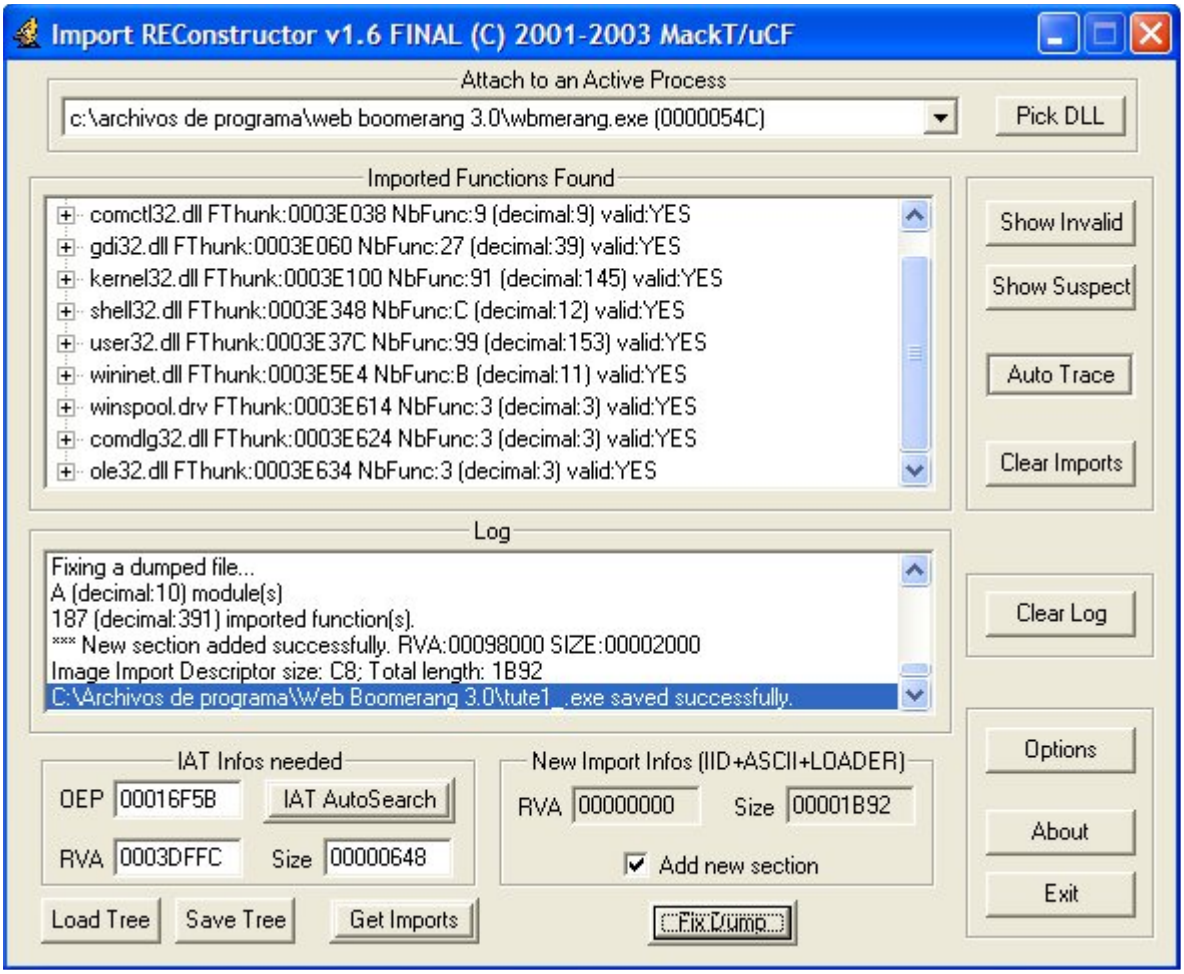
que son: uno en el OEP (416F5B), otro en la dirección del primer mágico (660038) y otro donde haremos el cambio de registros (660049), debemos reiniciar Olly y lanzar el script desde el EntryPoint ya que finciona bien, debemos además tener todas las excepciones desactivadas por eso, lo lanzamos y sale la nag, aceptamos y comienza a trabajar, vamos viendo como va evitando que salte en :660038 con las entradas del kernel y del user, cuando termine nos dará el típico mensajito de "script finished" que aceptamos para quedar detenidos en el OEP, jeje… y con la IAT parcialmente resuelta, aquél GetProcAddress que nos está dando guerra es la única entrada que queda por resolver, esto no es complicado ;-)

Abrimos el ImpRec, elegimos el programa de la lista "Attach to an Active Process" que se llamará "wbmerang.exe" le damos los datos correctos:

No creo que sea necesario volver a explicar este tema, supongo que estará claro a estas alturas, piacmos en "Get Imports" y nos quedó el GetProcAddress que ya esperábamos:

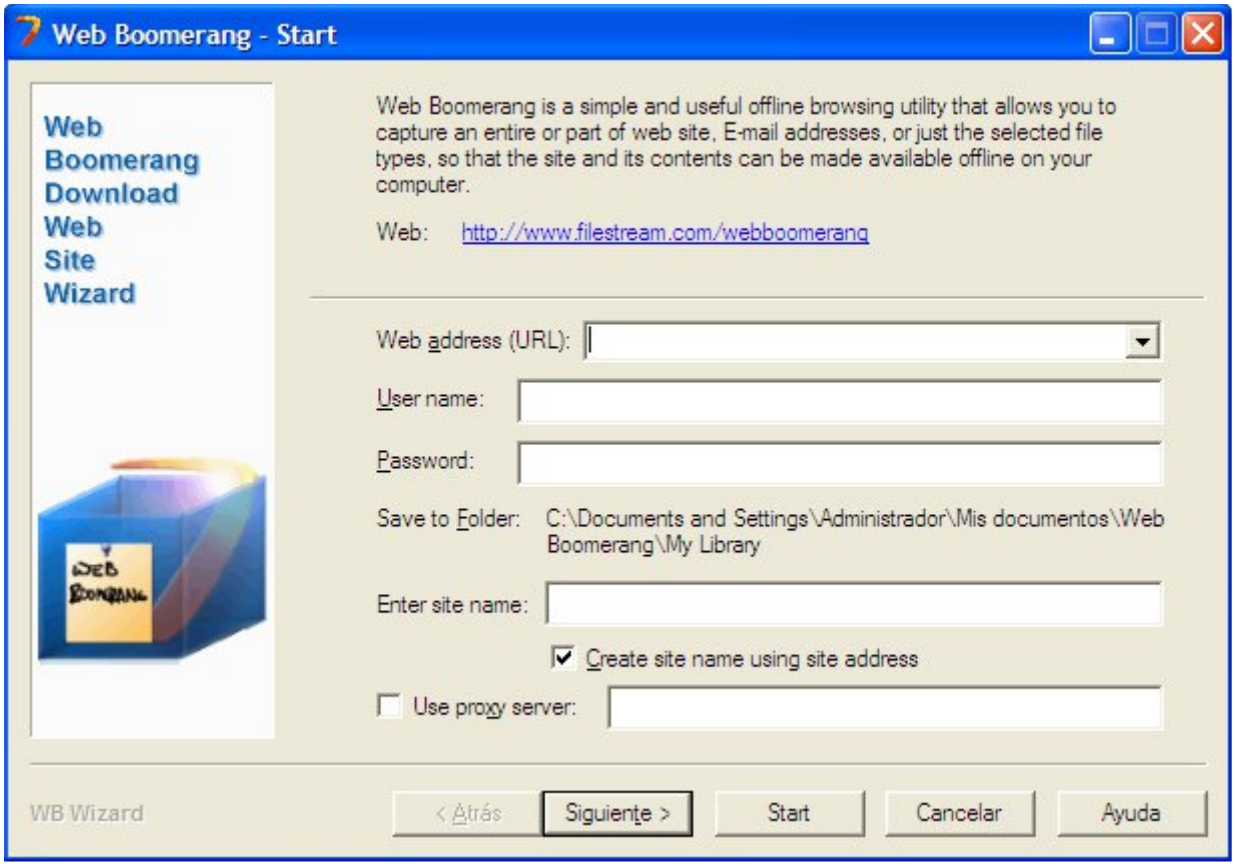


La reparamos con el propio Import y elegimos "Fix Dump" sí todavía no hemos dumpeado lo hacemos ahora mismo que estamos en el OEP parados, las opciones las de siempre, lo guardamos como tute1.exe para diferenciarlo del otro dumpeado tute.exe de antes, y lo reparamos con el bendito "Fix Dump" ;-)



Pues eso que salvó el archivo tute1_.exe que es el que tiene la IAT reparada ahora sí por completo, nos quedó por resolver GetProcAddress ya que la trata de distinta manera y nos daba malos resultados al intentar repararla mediante el script.

Sólo nos queda una cosa más: probarlo ;-DD lo ejecutamos y…



jeje.. también funciona, Visual Protect un packer antiguo que nos ha regalado un rato de diversión de eso se trata ;-) para que funcione bien debemos renombrar tute1_.exe como wbmerang.exe y al original ponerle antes otro nombre oara que no haya conflictos y lo sobrescriba ni nada, a esta aplicación se accede de dos formas: ejecutándola directamente desde el propio wbmerang.exe o bien desde el menú de wbb.exe:

Picando en Copy Wizard aparece la aplicación en pantalla, por eso decía renombrar el ejecutable tute1_.exe como originalmente se llama porque desde aquí se ejecuta wbmerang.exe. Dicho esto terminamos el tute de hoy xDD

Si a alguien le quedó alguna duda o no le quedó claro algo por supuesto puede escribirme a mi privado como siempre en:

syxe00@yahoo.es

Cualquier mensaje recibido será debidamente contestado me comprometo..!!

Y me despido con un saludo a todo CracksLatinoS sin excepción ninguna, para mi todos son estupendos y únicos, de cada uno aprendo algo diferente, mucha suerte y ojalá sigamos todos escribiendo, hasta la próxima…